

9 迭代器和组合模式

管理良好的 集合



有很多方法可以把对象堆起来成为集合。可以把它们放进数组、栈、列表、HashMap，随你挑。每一种都有自己的优劣，但在某些时候，你的客户想要遍历这些对象，而当他们这么做时，你希望把实现展示给客户吗？当然不！这太不专业了。好吧，你不必拿你的工作去冒险。本章中，你将看到如何能让客户遍历你的对象，而又无法窥视你是如何存储对象的。你也将学到如何创建某些对象超集合，一步跨越某些让人印象深刻的数据结构。除此之外，你还会学到一两件关于对象责任的事情。

爆炸性新闻：对象村餐厅和对象村煎饼屋合并

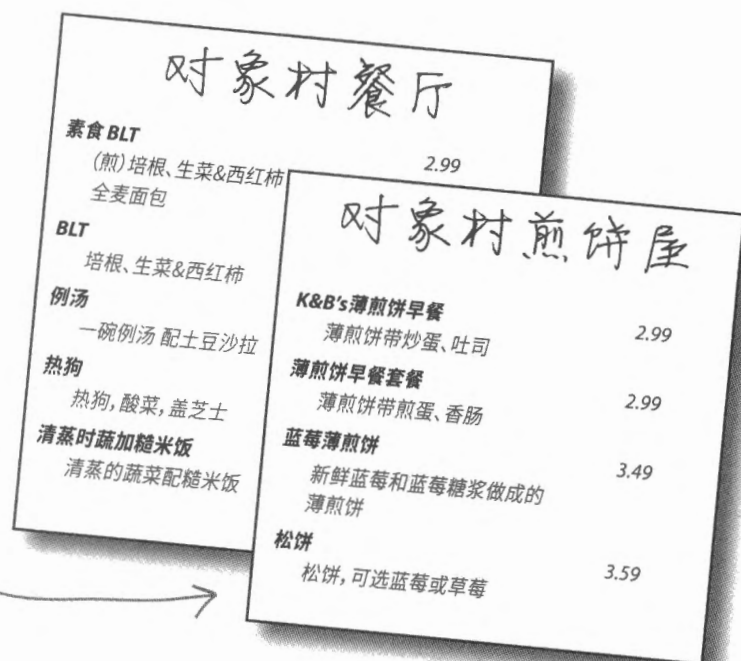
好消息！现在我们可以同一个地方，享用煎饼屋美味的煎饼早餐，以及好吃的餐厅午餐。但是好像有一点小麻烦……



检查菜单项

至少Lou和Mel都同意实现MenuItem。让我们检查每份菜单上的项，同时也看看其实现。

餐厅的菜单有许多午餐项，而煎饼屋的菜单包含的是早餐项。每个菜单项都有名称、描述和价格。



```
public class MenuItem {
    String name;
    String description;
    boolean vegetarian;
    double price;

    public MenuItem(String name,
                    String description,
                    boolean vegetarian,
                    double price)
    {
        this.name = name;
        this.description = description;
        this.vegetarian = vegetarian;
        this.price = price;
    }
}
```

菜单项包含名称、描述、指示是否为素食的标志和价格。将所有这些值传入构造器来初始化菜单项。

```
public String getName() {
    return name;
}

public String getDescription() {
    return description;
}

public double getPrice() {
    return price;
}

public boolean isVegetarian() {
    return vegetarian;
}
}
```

这些getter方法让你能够访问菜单项的各个字段。

Lou和Mel的菜单实现

现在让我们看看Lou和Mel在吵些什么。他们都在存储菜单项的方式上投入了很多时间和代码，还有很多其他代码依赖这些菜单项。



这是Lou的煎饼屋菜单实现。

```
public class PancakeHouseMenu {
    List<MenuItem> menuItems;
```

Lou使用一个ArrayList类存储他的菜单项。

```
public PancakeHouseMenu() {
    menuItems = new ArrayList<MenuItem>();
```

在构造器中，每个菜单项都被添加到ArrayList。

```
addItem("K&B's Pancake Breakfast",
        "Pancakes with scrambled eggs and toast",
        true,
        2.99);
```

每一个菜单项都有名称、描述、是否为素食项和价格。

```
addItem("Regular Pancake Breakfast",
        "Pancakes with fried eggs, sausage",
        false,
        2.99);
```

```
addItem("Blueberry Pancakes",
        "Pancakes made with fresh blueberries",
        true,
        3.49);
```

```
addItem("Waffles",
        "Waffles with your choice of blueberries or strawberries",
        true,
        3.59);
```

```
}
```

```
public void addItem(String name, String description,
                    boolean vegetarian, double price)
```

为了添加一个菜单项，Lou创建一个新的菜单项对象，给它传入每一个参量，然后把它添加到ArrayList。

```
{
    MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
    menuItems.add(menuItem);
}
```

getMenuItems()方法返回菜单项列表。

```
public ArrayList<MenuItem> getMenuItems() {
    return menuItems;
}
```

Lou还有一堆其他的菜单代码，它们都依赖于这个ArrayList实现，因此他不喜欢重写全部代码！

```
// 其他的菜单方法在这里
```

```
}
```



哈哈! **ArrayList**……我用的是真正的数组, 因此我能够控制菜单的最大尺寸。

```
public class DinerMenu {
    static final int MAX_ITEMS = 6;
    int numberOfItems = 0;
    MenuItem[] menuItems;
```

这是Mel的餐厅菜单实现。

Mel采用不同的做法; 他用一个数组类, 因此可以控制菜单的最大尺寸。

```
public DinerMenu() {
    menuItems = new MenuItem[MAX_ITEMS];

    addItem("Vegetarian BLT",
        "(Fakin') Bacon with lettuce & tomato on whole wheat", true, 2.99);
    addItem("BLT",
        "Bacon with lettuce & tomato on whole wheat", false, 2.99);
    addItem("Soup of the day",
        "Soup of the day, with a side of potato salad", false, 3.29);
    addItem("Hotdog",
        "A hot dog, with sauerkraut, relish, onions, topped with cheese",
        false, 3.05);
    // 这里添加一些其他餐厅菜单项
}
```

就跟Lou一样, Mel用addItem()辅助方法在构造器中创建菜单项。

```
public void addItem(String name, String description,
    boolean vegetarian, double price)
```

addItem()方法需要所有必要的参数来创建一个菜单项, 并实例化它。它也检查数组是否已经超出它的长度限制。

```
{
    MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
    if (numberOfItems >= MAX_ITEMS) {
        System.err.println("Sorry, menu is full! Can't add item to menu");
    } else {
        menuItems[numberOfItems] = menuItem;
        numberOfItems = numberOfItems + 1;
    }
}
```

Mel特别坚持让他的菜单保持在某个尺寸之内 (或许因此他就不必记住太多食谱)。

```
public MenuItem[] getMenuItems() {
    return menuItems;
}
```

和Lou一样, Mel还有一大堆代码依赖于菜单的这个数组实现。他忙着煮菜, 没空重写这么多代码!

// 其他的菜单方法在这里

有两种不同的菜单表现方式，这会带来什么问题？

为了了解为什么两种不同的菜单表现方式会让事情变得复杂，我们试着来实现一个使用这两个菜单的客户。假设你被餐厅和煎饼屋合并的新公司雇用，你的工作是创建一个Java版本的女招待（毕竟，这是对象村）。这个Java版本的女招待规格是：能按需为顾客打印定制的菜单，甚至不用问厨师就能告诉你是否某个菜单项是素食的。这可是一个创新！

我们来看看女招待的规格，然后看看怎样一步步实现她……

Java版本的女招待规格

Java版本的女招待：代号“Alice”

`printMenu()`

- 打印早餐和午餐菜单上的每一项

`printBreakfastMenu()`

- 只打印早餐项

`printLunchMenu()`

- 只打印午餐项

`printVegetarianMenu()`

- 打印所有素食菜单项

`isItemVegetarian(name)`

- 指定项的名称，如果该项是素食的话，返回true，否则返回false

女招待是Java版的



女招待的规格

实现规格: 我们的第一次尝试

我们先从实现printMenu()方法开始:

- 1 为了打印每份菜单上的所有项, 需要调用PancakeHouseMenu和DinerMenu的getMenuItem()方法, 来取得各自的菜单项。请注意, 两者的返回类型是不一样的:

```
PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
ArrayList<MenuItem> breakfastItems = pancakeHouseMenu.getMenuItems();

DinerMenu dinerMenu = new DinerMenu();
MenuItem[] lunchItems = dinerMenu.getMenuItems();
```

方法看起来一样,
但是调用返回的类型不同。

实现显露出来: 早餐项是在一个ArrayList中, 午餐项则是在一个数组中。

- 2 现在, 为了打印PancakeHouseMenu的项, 我们用循环遍历breakfastItems ArrayList内的项。为了打印餐厅的项, 用循环遍历数组:

```
for (int i = 0; i < breakfastItems.size(); i++) {
    MenuItem menuItem = breakfastItems.get(i);
    System.out.print(menuItem.getName() + " ");
    System.out.println(menuItem.getPrice() + " ");
    System.out.println(menuItem.getDescription());
}

for (int i = 0; i < lunchItems.length; i++) {
    MenuItem menuItem = lunchItems[i];
    System.out.print(menuItem.getName() + " ");
    System.out.println(menuItem.getPrice() + " ");
    System.out.println(menuItem.getDescription());
}
```

现在, 我们不得不实现两个不同的循环, 遍历菜单项的这两个实现……

……处理ArrayList的循环……

……处理数组的循环。

- 3 如何实现Waitress中的其他方法, 是这一幕的变体。我们总是需要处理两个菜单, 并且用两个循环遍历它们的项。如果再并购另一家有不同实现的餐厅, 我们就有三个循环。



削尖你的铅笔

基于我们的printMenu()实现，下列哪些是对的？

- ☐ A. 我们在针对PancakeHouseMenu和DinerMenu的具体实现编码，而不是针对接口。
- ☐ B. 女招待没有实现Java女招待API，所以她没有遵守标准。
- ☐ C. 如果我们决定从使用DinerMenu切换到另一种菜单，此菜单的菜单项列表是用哈希表实现的，那么我们不得不修改女招待中的很多代码。
- ☐ D. 女招待需要知道每个菜单如何表达内部的菜单项集合，这违反了封装。
- ☐ E. 我们有重复的代码，printMenu()方法需要两个分离的循环，来遍历两种不同的菜单。如果我们加上第三种菜单，我们又得加一个循环。
- ☐ F. 这个实现没有基于MXML（Menu XML），所以没有办法互操作。

然后怎样？

Mel和Lou让我们很为难。他们都不想改变他们的实现，因为这意味着要重写各自菜单类的很多代码。但如果没有人退让，我们只能实现一个难以维护和扩展的女招待。

如果我们能够找到办法，让他们的菜单实现同一个接口，该有多好（菜单是相似的，除了getItem()方法的返回类型不同）。这样，我们就可以把女招待代码中的具体引用最小化，也有希望摆脱遍历两个菜单所需的多个循环。

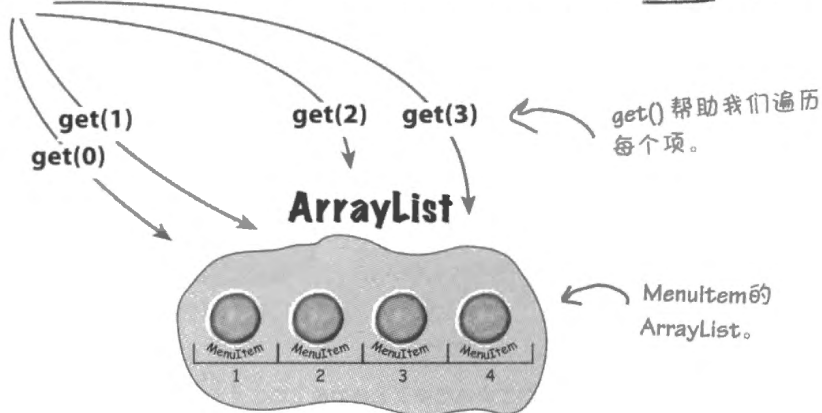
听起来很棒吧？但要怎么做呢？

可以封装遍历吗？

如果我们从本书中学到一件事情，那就是封装变化。很明显，在这里发生变化的是：菜单返回的不同对象集合所引起的遍历。但我们能封装这个吗？让我们来完成这个想法……

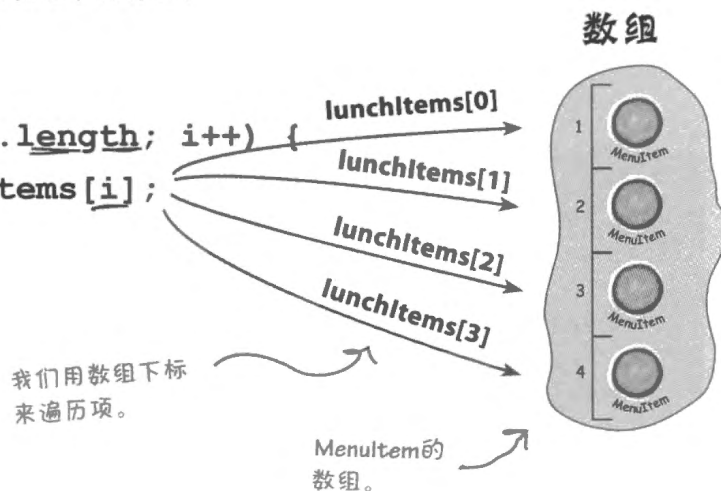
❶ 要遍历早餐项，我们用ArrayList的size()和get()方法：

```
for (int i = 0; i < breakfastItems.size(); i++) {
    MenuItem menuItem = breakfastItems.get(i);
}
```



❷ 要遍历午餐项，我们用数组的length字段以及数组下标记号：

```
for (int i = 0; i < lunchItems.length; i++) {
    MenuItem menuItem = lunchItems[i];
}
```



3 现在我们创建一个对象，我们称为迭代器（Iterator），它封装遍历对象集合的方式。我们在ArrayList上试试：

我们找breakfastMenu要一个MenuItem的迭代器。

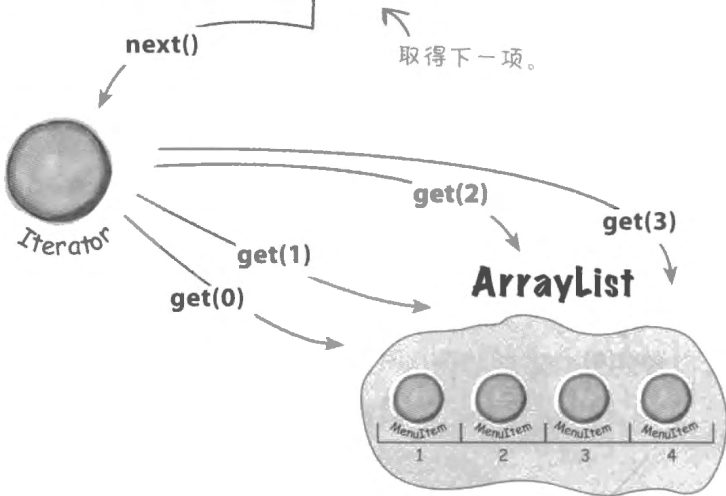
```
Iterator iterator = breakfastMenu.createIterator();
```

```
while (iterator.hasNext()) {  
    MenuItem menuItem = iterator.next();  
}
```

当还有更多项时……

取得下一项。

客户只需要调用hasNext()和next()。在幕后，迭代器调用ArrayList的get()。



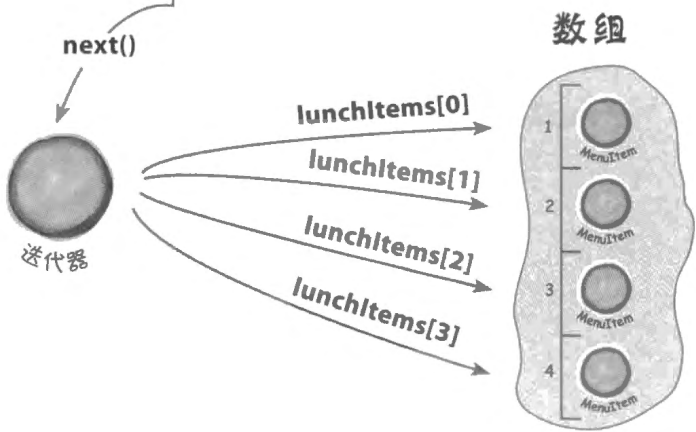
4 在数组上也试试：

```
Iterator iterator = lunchMenu.createIterator();
```

```
while (iterator.hasNext()) {  
    MenuItem menuItem = iterator.next();  
}
```

哇哦，这个代码和breakfastMenu的代码完全一样。

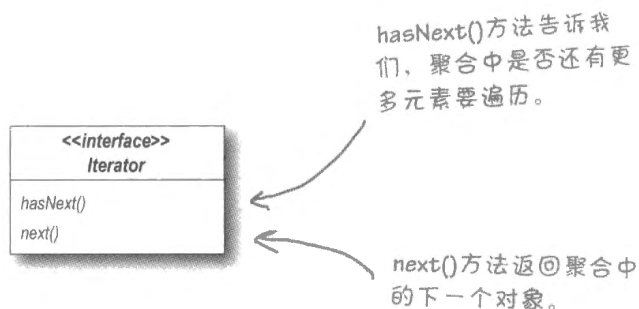
这里的情况也一样：客户只需要调用hasNext()和next()，在幕后，迭代器使用数组的下标。



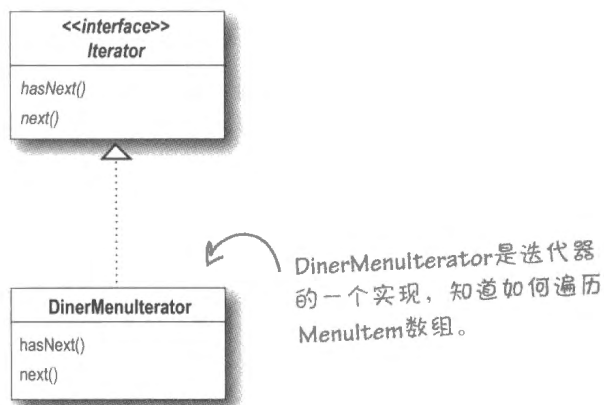
认识迭代器模式

好，看起来我们封装迭代的计划已经奏效了，你大概也已经猜到，它是一个设计模式，称为迭代器模式（Iterator Pattern）。

关于迭代器模式，你需要知道的第一件事情是，它依赖于一个名为迭代器（Iterator）的接口。迭代器接口可能是这样的：



现在，一旦我们有了这个接口，就可以为各种对象集合实现迭代器：数组、列表、哈希表……随你喜欢。如果我们想要为用在DinerMenu中的数组实现迭代器，可以这样：



让我们继续实现这个迭代器，并把它整合进DinerMenu，来看看它如何工作……

当我们说“集合”（collection）的时候，我们指的是一群对象。它们的存储方式可能差别很大，例如列表、数组、哈希表，但它们依然是集合。有时候也称为聚合（aggregate）。



给DinerMenu添加一个迭代器

为了给DinerMenu添加一个迭代器，我们需要先定义迭代器接口：

```
public interface Iterator {  
    boolean hasNext();  
    MenuItem next();  
}
```

这是我们的两个方法：

`hasNext()`方法会返回一个布尔值，表示是否还有更多元素需要遍历……

……`next()`方法返回下一个元素。

现在我们需要实现一个具体的迭代器，为餐厅菜单服务：

```
public class DinerMenuIterator implements Iterator {  
    MenuItem[] items;  
    int position = 0;  
  
    public DinerMenuIterator(MenuItem[] items) {  
        this.items = items;  
    }  
  
    public MenuItem next() {  
        MenuItem menuItem = items[position];  
        position = position + 1;  
        return menuItem;  
    }  
  
    public boolean hasNext() {  
        if (position >= items.length || items[position] == null) {  
            return false;  
        } else {  
            return true;  
        }  
    }  
}
```

实现迭代器接口。

`position`维护数组遍历的当前位置。

构造器需要待遍历的菜单项数组作为参数。

`next()`方法返回数组内的下一项，并递增`position`。

`hasNext()`方法检查我们是否已经看过数组的所有元素。如果还有元素待遍历，则返回`true`。

因为用的是有最大长度的数组，我们需要检查是否处于数组末尾，也要检查下一项是否为空，如果为空，就表示没有其他项了。

用迭代器改写餐厅菜单

OK，我们有了迭代器。是时候把它用在DinerMenu上了。我们只需加入一个方法来创建一个DinerMenuIterator，并把它返回给客户：

```
public class DinerMenu {
    static final int MAX_ITEMS = 6;
    int numberOfItems = 0;
    MenuItem[] menuItems;
```

// 构造器在这里

// addItem在这里

```
public MenuItem[] getMenuItems() {
    return menuItems;
}
```

我们不再需要getMenuItems()方法。事实上，我们不想要这个方法，因为它暴露了我们的内部实现。

```
public Iterator createIterator() {
    return new DinerMenuIterator(menuItems);
}
```

这是createIterator()方法。它从菜单项数组创建一个DinerMenuIterator，并把它返回给客户。

// 菜单的其他方法在这里

```
}
```

我们返回迭代器接口。客户不需要知道DinerMenu是如何维护菜单项的，也不需要知道DinerMenuIterator如何实现。它只需要使用这个迭代器来遍历菜单项。



自行实现PancakeHouseIterator，并作必要的修改以整合进PancakeHouseMenu。

修正女招待的代码

现在，我们需要把迭代器代码整合进Waitress类。我们应该能去除一些过程中的冗余。整合相当直截了当：首先创建一个printMenu()方法，传入一个迭代器作为参数，然后对每个菜单使用createIterator()方法来取得迭代器，并把它传入新的方法。

```
public class Waitress {
    PancakeHouseMenu pancakeHouseMenu;
    DinerMenu dinerMenu;

    public Waitress(PancakeHouseMenu pancakeHouseMenu, DinerMenu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }

    public void printMenu() {
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator dinerIterator = dinerMenu.createIterator();

        System.out.println("MENU\n----\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }

    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }

    // 其他方法在这里
}
```

经迭代器改良
成新版的我。

在构造器中，女招待以两个
菜单作为参数。

printMenu()方法现在
创建两个迭代器，一
个菜单一个……

然后调用重载的
printMenu()，将迭代器
传入。

测试是否还有
更多项。

取得下
一项。

重载的
printMenu()方法
用迭代器来遍历
菜单项并打印。

使用项以取得名
称、价格和描述，
并打印。

注意，我们一个循环
就搞定。



测试我们的代码

是时候测试每一样东西了。让我们写一些测试程序，
然后看看女招待如何工作……

```
public class MenuTestDrive {
    public static void main(String args[]) {
        PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
        DinerMenu dinerMenu = new DinerMenu();

        Waitress waitress = new Waitress(pancakeHouseMenu, dinerMenu);

        waitress.printMenu();
    }
}
```

首先我们创建了新的菜单。

然后我们创建了一个女招待，并将菜单传给她。

然后打印。

执行结果……

File Edit Window Help

```
% java DinerMenuTestDrive
```

```
MENU
```

```
----
```

```
BREAKFAST
```

```
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs and toast
```

```
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage
```

```
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries
```

```
Waffles, 3.59 -- Waffles with your choice of blueberries or strawberries
```

```
LUNCH
```

```
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat
```

```
BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat
```

```
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad
```

```
Hot Dog, 3.05 -- A hot dog, with sauerkraut, relish, onions, topped with cheese
```

```
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice
```

```
Pasta, 3.89 -- Spaghetti with marinara sauce, and a slice of sourdough bread
```

```
%
```

首先我们遍历
煎饼屋菜单。

然后是午餐
菜单，所
用的遍历代
码相同。

目前为止我们做了什么？

首先，我们让对象村的厨师们非常快乐。他们可以保持他们自己的实现又可以摆平差别。只要我们给他们PancakeHouseMenuIterator和DinerMenuIterator，他们只需要添加一个createIterator()方法，就搞定了。

在这个过程中，我们也帮了我们自己。女招待以后会更容易维护和扩展。我们过一遍我们到底做了什么，并思考其后果。



难以维护的 女招待实现

菜单封装得不好，我们可以看到餐厅用的是ArrayList，而煎饼屋用的是数组。

需要两个循环来遍历菜单项。

女招待捆绑到具体类（MenuItem[]和ArrayList）。

女招待捆绑到两个不同的具体菜单类，尽管它们的接口大致相同。

时髦的 由迭代器赋能的新女招待

菜单的实现现在被封装。女招待不知道菜单如何持有菜单项的集合。

只要实现迭代器，我们只需一个循环，就可以多态地处理任何项的集合。

女招待现在只使用一个接口（迭代器）。

现在菜单的接口完全一样，哎呀，我们还是没有一个共同的接口，这意味着女招待仍然绑定到两个具体的菜单类。这个最好修正一下。

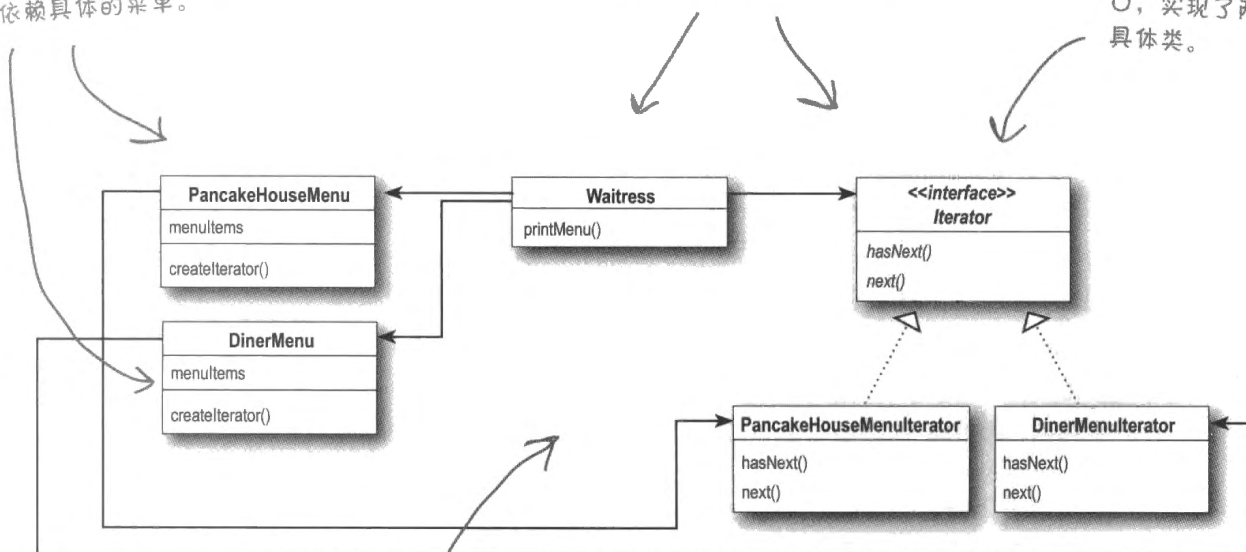
回顾我们目前的设计……

在把所有都搞定之前，让我们“鸟瞰”一下目前的设计。

这两个菜单都实现一模一样的方法集，但是并没有实现相同的接口，我们会修正这一点，好让女招待不会依赖具体的菜单。

这个迭代器让女招待能够从具体类的实际实现解耦。她不需知道菜单是使用数组、ArrayList还是便利贴来实现。她只关心她能够取得迭代器来做她的遍历。

我们现在使用一个共同的迭代器接口，实现了两个具体类。



请注意，迭代器让我们能够遍历聚合中的元素，而不强迫聚合塞给自己的接口一堆方法，来支持遍历各元素。它也使得迭代器的实现可以放在聚合的外面，换句话说，我们封装了迭代。

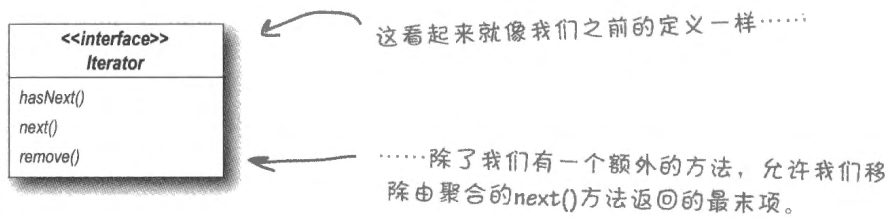
PancakeHouseMenu和**DinerMenu**都实现了新的`createIterator()`方法，它们负责为各自的菜单项实现创建迭代器。

做些改进……

OK，我们已经知道PancakeHouseMenu和DinerMenu的接口完全一样，但还没有为它们定义一个共同的接口。因此，我们打算这样做，让Waitress更整洁一些。

你可能会纳闷为什么我们不使用Java的Iterator接口，是因为这样你可以看到如何从头建造一个迭代器。既然目的已经达到，我们打算转到使用Java的Iterator接口，因为实现这个接口和实现自己做的接口比起来，有很多好处。什么样的好处？你一会就会看到。

首先，我们来看看 java.util.Iterator 接口：



做起来很容易，我们只需要改变PancakeHouseMenuIterator和DinerMenuIterator所扩展的接口，对吧？差不多……实际上甚至更简单。其实不只java.util有自己的迭代器接口，ArrayList也有一个iterator()方法，此方法返回一个迭代器。换句话说，我们绝对不需要为ArrayList实现自己的迭代器。然而，我们仍然需要为DinerMenu实现一个迭代器，因为它依赖于数组，而数组不支持iterator()方法。

没有蠢问题

问： 如果我不想提供移除潜在对象集合中的某些东西的能力，怎么办？

答： remove()方法被认为是可选的。你不一定要提供移除功能，但是，你应该提供，因为它是Iterator接口的一部分。如果你不打算在迭代器中允许remove()，可以抛出运行时异常java.lang.UnsupportedOperationException。Iterator的API文档说明，这个异常可以从remove()抛出，任何良好的客户程序，在调用remove()方法时，会检查这个异常。

问： 在多线程的情况下，可能会针对同一个对象集合使用不同的迭代器。remove()会有怎样的行为？

答： 如果你在集合中遍历时，集合变化了，此时remove()方法的行为怎样，没有说明。因此，当并发访问一个集合时，你应该小心设计你自己的多线程代码。

用java.util.Iterator来清理

我们从PancakeHouseMenu开始，让它改用java.util.Iterator，这很容易。
我们只要删除PancakeHouseMenuIterator类，然后在PancakeHouseMenu
代码的顶部，添加import java.util.Iterator，再修改一行代码：

```
public Iterator<MenuItem> createIterator() {  
    return menuItems.iterator();  
}
```

现在我们不创建自己的迭代器了，只是调用菜单项ArrayList的iterator()方法（一会还会谈到这一点）。

这样PancakeHouseMenu就完成了。

现在，我们需要做一些修改，让DinerMenu和java.util.Iterator能一起工作。

```
import java.util.Iterator;
```

首先我们导入java.util.Iterator，我们要实现这个接口。

```
public class DinerMenuIterator implements Iterator<MenuItem> {  
    MenuItem[] items;  
    int position = 0;
```

```
    public DinerMenuIterator(MenuItem[] items) {  
        this.items = items;  
    }
```

```
    public MenuItem next() {  
        //实现在这里  
    }
```

```
    public boolean hasNext() {  
        //实现在这里  
    }
```

当前的实现没有变化。

```
    public void remove() {  
        throw new UnsupportedOperationException  
            ("You shouldn't be trying to remove menu items.");  
    }
```

记住，remove()方法在迭代器接口中是可选的。让女招待移除菜单项其实没有什么意义，所以如果她要试一试，我们只是抛出一个异常。

就快好了……

现在我们只需要给菜单一个共同的接口，然后再稍微改一下Waitress。Menu接口相当简单，最终我们可能要给它添加更多的方法，例如addItem()，但是目前，我们还是让厨师控制他们的菜单，不把那些方法放在公开接口中：

```
public interface Menu {
    public Iterator<MenuItem> createIterator();
}
```

这是一个简单接口，让客户取得一个菜单项迭代器。

现在，我们需要给PancakeHouseMenu和DinerMenu的类定义添加implements Menu，并更新Waitress类：

```
import java.util.Iterator;
```

现在Waitress也使用java.util.Iterator。

```
public class Waitress {
    Menu pancakeHouseMenu;
    Menu dinerMenu;
```

我们需要替换具体Menu类为Menu接口。

```
    public Waitress(Menu pancakeHouseMenu, Menu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }
```

```
    public void printMenu() {
        Iterator<MenuItem> pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator<MenuItem> dinerIterator = dinerMenu.createIterator();
        System.out.println("MENU\n----\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }
```

```
    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }
```

这部分没有修改。

```
    // 其他方法在这里
```

```
}
```

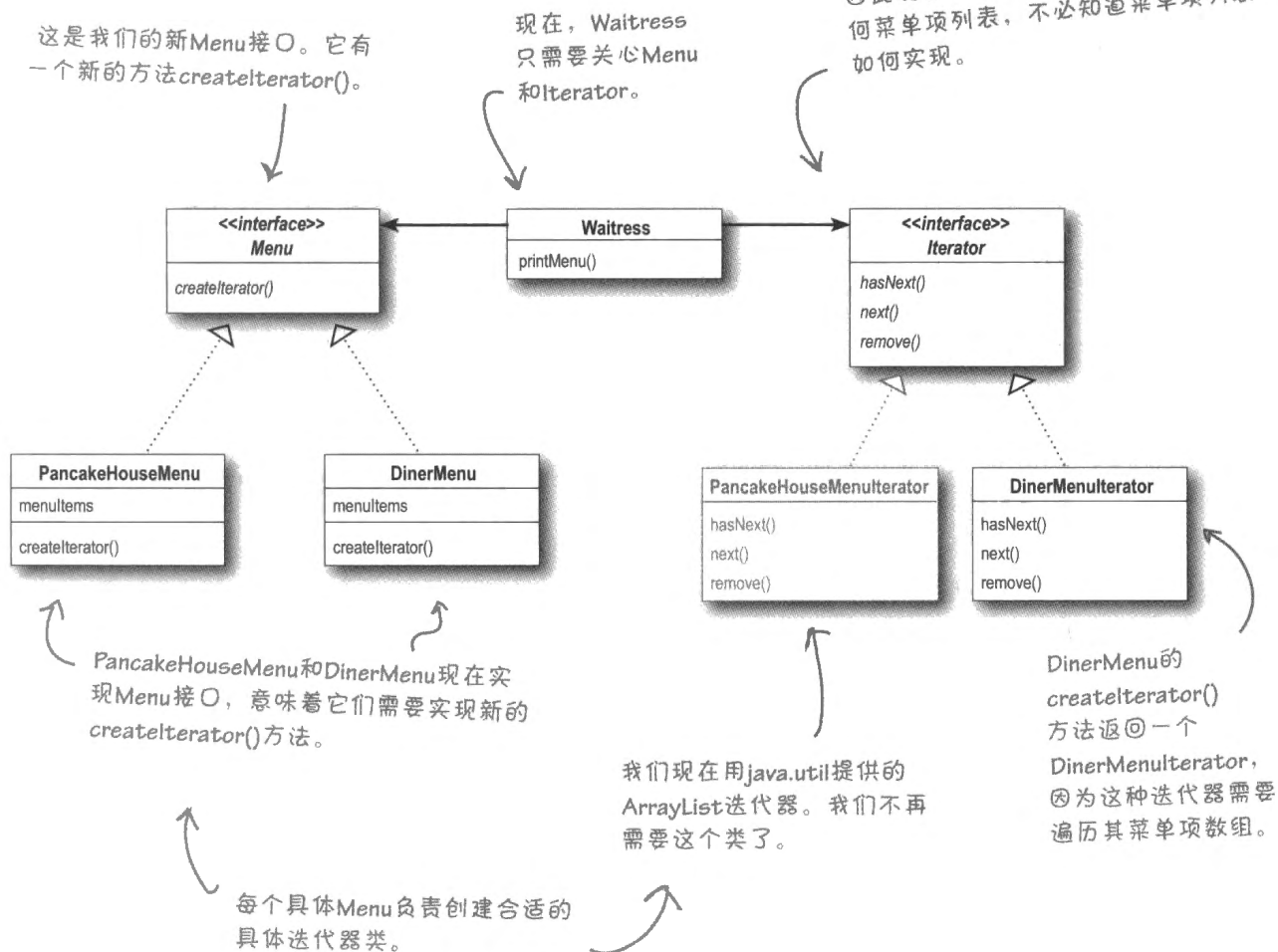
这样做对我们有什么好处？

PancakeHouseMenu和DinerMenu类实现了一个接口Menu。这使得Waitress可以用接口来引用每一个菜单对象，而不是引用具体类。因此，通过“针对接口编程，而不是针对实现编程”，我们减少了Waitress和具体类之间的依赖。

这解决了Waitress依赖具体菜单的问题。

这个新的Menu接口有一个方法，createIterator()，这个方法被PancakeHouseMenu和DinerMenu实现。每个菜单类负责创建适合菜单项内部实现的具体迭代器。

我们将Waitress从菜单的实现解耦，因此现在我们可以用迭代器来遍历任何菜单项列表，不必知道菜单项列表如何实现。



定义迭代器模式

你已经看到如何用你自己的迭代器实现迭代器模式。你也看到了Java如何在某些面向集合的类（ArrayList）中支持迭代器。现在，是时候来看看这个模式的官方定义了：

迭代器模式提供一种方式，可以访问一个聚合对象中的元素而又不暴露其潜在的表示。

这很有意义，这个模式给了你一种遍历聚合元素而又不必知道事物在底层如何表达的方式。你已经在前面的两个菜单实现中看到了这一点。而且在设计中使用迭代器的影响也是很大的，一旦你有统一的方式来访问所有聚合对象的元素，你就可以编写多态的代码，和任意聚合一起使用——就像前面的printMenu()方法，这个方法不在意菜单项是放在数组还是ArrayList（或者其他能创建迭代器的东西）中，只要它能持有迭代器。

另一个对于你的设计所造成的重要影响是，迭代器模式拿走了遍历元素的责任，把它交给迭代器对象，而不是聚合对象。这不仅使得聚合的接口和实现更简单，它还从聚合移除了遍历的责任，让聚合专注于应该专注的事情（管理对象集合）。

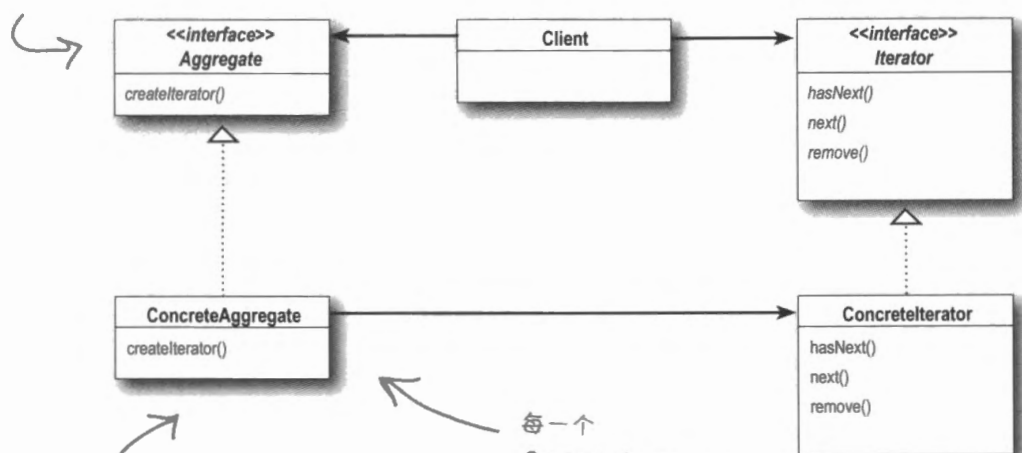
迭代器模式让我们遍历聚合元素，而不暴露其潜在实现。

它也把遍历的任务放在迭代器对象上，而不是聚合上，这样就简化了聚合的接口和实现，让责任放在合适的地方。

迭代器模式的结构

我们来看看类图，图中描述了上下文的所有片段：

聚合有一个共同的接口，这对客户来说是很方便的：它把客户从集合对象的实现解耦。



Iterator接口提供了所有迭代器都必须实现的接口。它提供了一个遍历集合元素的方法集。在这里，我们用的是java.util.Iterator。如果你不想用Java的迭代器接口，可以创建自己的接口。

ConcreteAggregate有一个对象的集合，并实现一个方法，该方法返回集合的迭代器。

每一个ConcreteAggregate负责实例化一个能够遍历其对象集合的ConcreteIterator。

ConcreteIterator负责管理当前遍历的位置。

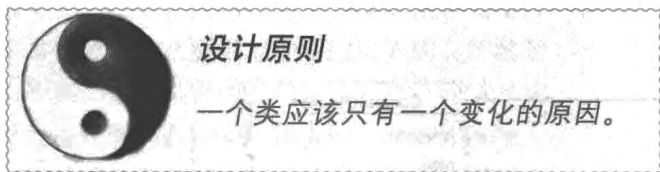


迭代器模式的这张类图，看起来很像我们所学过的另一个模式，你知道是哪个模式吗？提示：子类决定要创建哪个对象。

单一责任原则

如果允许我们的聚合实现它们内部的集合以及相关的操作和遍历的方法，会怎么样？我们已经知道这会增加聚合中的方法数目，但又怎样？为什么这样做不好？

想知道为什么，首先你需要认识到，当我们允许一个类不但要关心自己的业务（管理某种聚合），而且要承担更多的责任（像遍历）时，类就有了两个发生变化的原因。两个？对的，两个：如果集合变化，它要变；如果遍历方式变化，它也要变。因此，再一次，我们的老朋友“变化”处在另一个设计原则的中心：



因为修改代码，很容易给各种问题提供了潜入的机会，所以我们知道要避免类里面的变化。有两个变化的原因，将来类变化的机会增加了，当它变化时，会影响你的设计中的两个方面。

解决方案？这个原则指引我们，只把一个责任分配给一个类而且仅一个类。

没错，这好像很容易，但再想一想没那么容易，分离设计中的责任，是最困难的事情之一。我们的大脑太擅长看着一组行为并把它们集中在一起，即使它们其实属于两个或多个的责任。想要成功的唯一方法，就是努力不懈地检查你的设计，观察当系统成长时有没有迹象显示某个类变化的原因超出一个。

类的每个责任都是一个潜在变化的区域。超过一个责任，意味着超过一个变化的区域。

这个原则指导我们要让每个类保持单一责任。



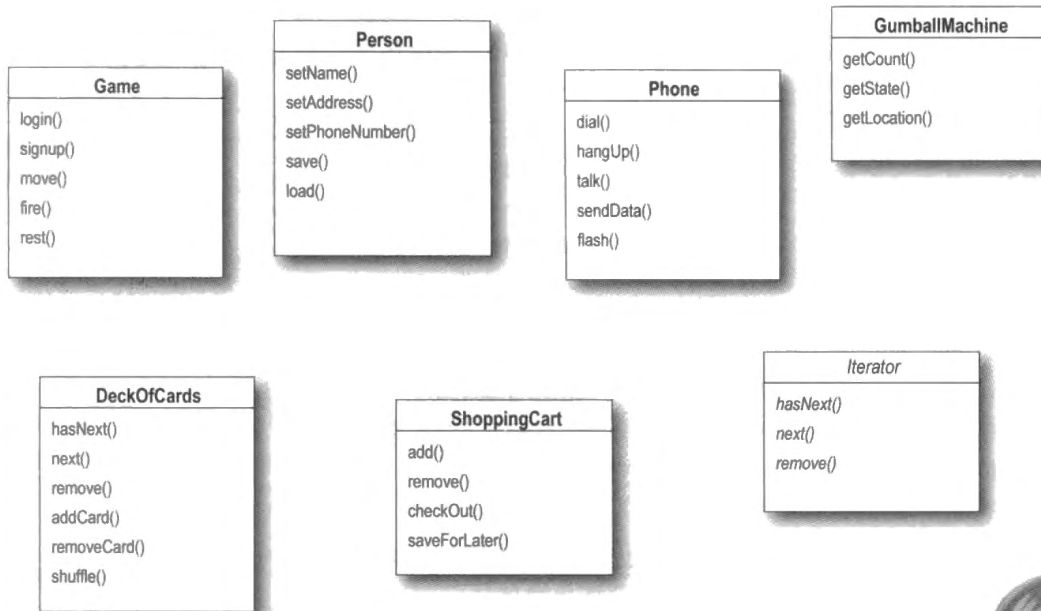
内聚（cohesion）这个术语你应该听过，它用来度量一个类或模块完成单个目的或者责任时有多紧密。

当一个模块或一个类围绕一组相关功能来设计时，我们说它具有高内聚；反之，当它围绕一组相关不功能来设计时，我们说它具有低内聚。

比起单一责任原则，内聚是更普遍的概念，不过两者关系密切。遵守此原则的类容易有高内聚，比承担多个责任的低内聚的类更有可维护性。



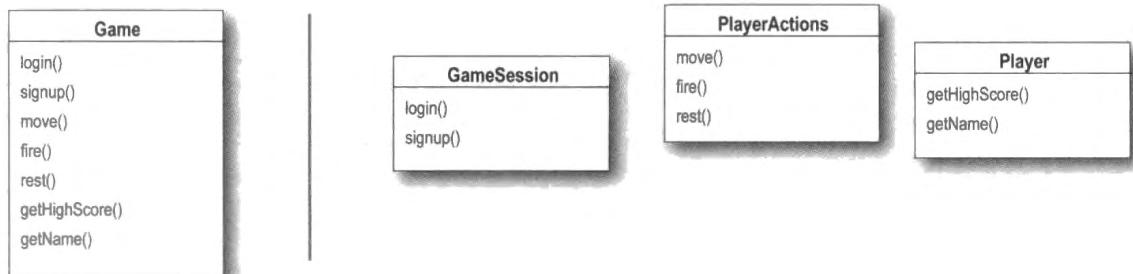
研究这些类，并找出其中哪些类具有多重责任。



安全帽区！注意落物伤人。



判断这些类是低内聚还是高内聚。



没有蠢问题

问：我看到其他书上展示的迭代器类图，方法是first()、next()、isDone()和currentItem()。为什么这些方法不一样？

答：这些是以前用过的“经典”方法名。随着时间的推移，这些名称已经变化。我们现在在java.util.Iterator中用的是next()、hasNext()，甚至remove()。

我们来看看这些经典的方法。在java.util中，next()和currentItem()被合并进一个方法。isDone()方法变成了hasNext()；但first()没有对应的方法，这是因为在Java中，需要开始遍历时，我们倾向于取得一个新的迭代器。不过，你可以看到，这些接口其实差别很小。事实上，你还可以给迭代器添加大把行为，remove()方法就是java.util.Iterator延伸的一个例子。

问：我听说过“内部的”迭代器和“外部的”迭代器。它们是什么？我们前面例子中实现的是哪种？

答：我们实现的是外部的迭代器，意味着，客户通过调用next()取得下一个元素，以此控制遍历。内部迭代器由迭代器自己控制。在这种情况下，因为迭代器在元素之间游走，你不得不告诉迭代器在游走时对这些元素做什么事情，也就是说，你要有办法把操作传给迭代器。内部迭代器比外部迭代器弹性更小，因为客户无法控制遍历。然而，有人会争论说，内部的迭代器更容易使用，因为只要把一个

操作交给它们并让它们遍历，它们就会帮你做所有工作。

问：迭代器可以被实现成向后移动吗，就像向前移动一样？

答：肯定的。在这种情况下，你可能要添加两个方法，一个方法取得前一个元素，另一个告诉你什么时候处在集合的开头。Java的Collection Framework提供另一种类型的迭代器接口，称为ListIterator。这个迭代器给标准的迭代器接口添加了previous()和一些其他的方法。任何实现了List接口的集合，都支持这样的迭代器。

问：像哈希表这样没有内在次序的集合，谁来定义遍历次序？

答：迭代器不隐含次序。潜在的集合可以是没有次序的，例如哈希表或bag，甚至包含重复内容。因此，排序和潜在集合的属性及实现有关。一般来说，除非集合的文档有说明，否则你不应该对次序做任何假设。

问：你说我们可以用迭代器写出“多态的代码”，能进一步解释一下吗？

答：当我们编写以迭代器作为参数的方法时，我们就是在用多态的迭代。意思是，我们正在创建可以遍历任意集合的代码，只要这个集合支持迭代器。我们不在意这个集合如何实现，但依然可以编码来遍历它。

问：如果我在用Java，我不想总是用java.util.Iterator接口，那我能把自己的迭代器实现和已经在用Java迭代器的类一起使用吗？

答：可能可以。如果你有一个共同的迭代器接口，那么混合并匹配你自己的聚合和Java聚合（例如ArrayList和Vector）会更容易。但记住，如果你需要给聚合的迭代器接口添加功能，你总是可以扩展迭代器接口。

问：我看到Java有一个Enumeration（枚举）接口，它有没有实现迭代器模式？

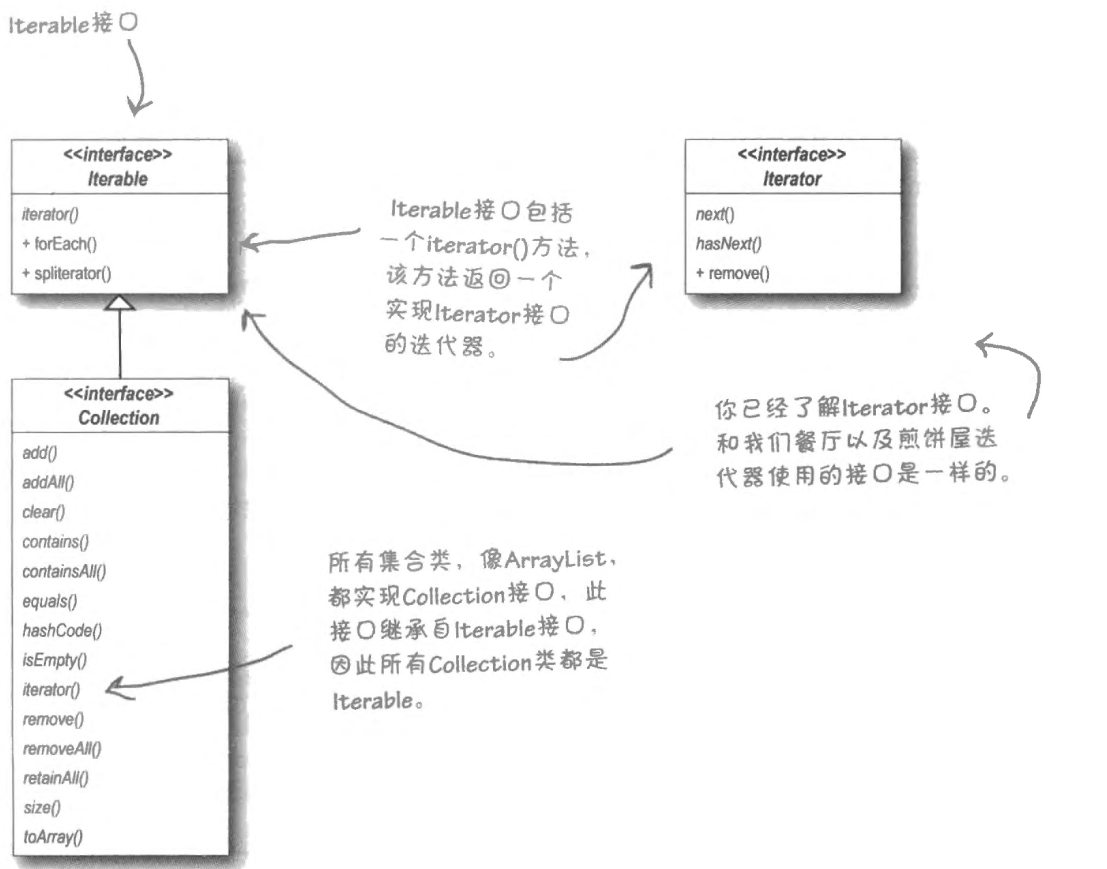
答：我们曾经在适配器模式那一章（第7章）谈到这个接口，还记得吗？java.util Enumeration是一个更老的迭代器实现，已经被java.util.Iterator取代。Enumeration有两个方法：hasMoreElements()，对应于hasNext()；nextElement()，对应于next()。不过，你可能更想用Iterator而不是Enumeration，因为更多Java类支持Iterator。如果你需要把一个转换成另一个，请再复习第7章中关于实现Enumeration和Iterator的适配器的内容。

问：使用Java的enhanced for loop和迭代器有关吗？

答：好问题！有关。要回答这个问题，我们需要理解另一个接口，就是Java的Iterable接口。现在正是时候开始……

认识Java的Iterable接口

你已经了解Java最新的Iterator接口，但还有另一个接口你需要认识：Iterable。Iterable接口可以被Java的每个集合类型实现。你猜怎么着？在代码中使用ArrayList时，你已经在使用这个接口。我们来看看Iterable接口：



如果一个类实现Iterable，我们知道这个类实现了一个iterator()方法。这个方法返回一个实现Iterator接口的迭代器。这个接口也包括一个缺省的forEach()方法，可以用作遍历集合的另一种方式。除了以上所有这些，通过enhanced for loop，Java甚至提供一些很不错的迭代语法糖。我们来看看是怎么回事。

Iterable接口也包括spliterator()方法。该方法甚至提供高级的方式来遍历集合。

Java的enhanced for loop

我们拿一个对象来说吧，该对象的类实现了Iterable接口……例如用于煎饼屋菜单项的ArrayList集合：

```
List<MenuItem> menuItems = new ArrayList<MenuItem>();
```

我们可以用之前的方法来遍历ArrayList：

```
Iterator iterator = menu.iterator();
while (iterator.hasNext()) {
    MenuItem menuItem = iterator.next();
    System.out.print(menuItem.getName() + ", ");
    System.out.print(menuItem.getPrice() + " -- ");
    System.out.println(menuItem.getDescription());
}
```

这是我们之前遍历集合的方式，用迭代器以及hasNext()和next()方法来遍历。

或者，既然我们知道ArrayList是Iterable，我们可以用Java的enhanced简写：

```
for (MenuItem item: menu) {
    System.out.print(menuItem.getName() + ", ");
    System.out.print(menuItem.getPrice() + " -- ");
    System.out.println(menuItem.getDescription());
}
```

这里我们可以给显式的迭代器分配hasNext()和next()方法。



看起来，使用Iterator真是个好办法，代码更简单，不再调用hasNext()或next()方法。那么，我们能用Iterable和enhanced for loop来为两个菜单重新编写Waitress的代码吗？

没那么快：数组不是Iterable

我们有些坏消息：餐馆使用数组作为菜单的基础，可能不是最好的决定。事实证明，数组不是Java的集合，所以它们不实现Iterable接口。这样的话，我们就不能轻易地把Waitress的代码整合到一个方法，以一个Iterable为参数，既把它用于煎饼屋的早餐项又把它用于餐厅的午餐项。如果你尝试改变Waitress的printMenu()方法为接收一个Iterable而不是一个Iterator，并用for-each循环而不是Iterator API，像这样：

```
public void printMenu(Iterable<MenuItem> iterable) {
    for (MenuItem menuItem : iterable) {
        // print menuItem
    }
}
```

这只适用于我们煎饼屋菜单的ArrayList。

当你尝试把午餐项数组传到printMenu()时会得到一个编译错误：

```
printMenu(lunchItems);
```

编译错误！数组不是Iterable。

原因还是数组不实现Iterable接口。

如果你保留Waitress代码的两个循环，我们又回到起点：Waitress又一次依赖我们用来存储菜单的聚合类型，她有着重复的代码：一个循环针对ArrayList，一个循环针对数组。

那么我们怎么做？有很多种方式来解决这个问题，但是它们都有点节外生枝，需要重构我们的代码。毕竟本章是关于迭代器模式，而不是关于Java的Iterable接口。不过好消息是你知道Iterable，你也知道它和Java的Iterator接口以及迭代器模式的关系。因此，我们继续往下走，因为我们已经有了非常棒的实现，尽管我们没有利用一点Java的循环语法糖优势。



Serious Coding

你可能注意到了Iterable菜单的forEach()方法。它用作Java的enhanced for loop的基础，但你也可以直接把它和Iterables一起使用。以下是用法：

这是一个Iterable，本例中是煎饼屋的菜单项ArrayList。

我们正调用forEach()……

……传递一个以菜单项为参数的Lambda表达式，只为了打印。

```
breakfastItems.forEach(item -> System.out.println(item));
```

所以这个代码会打印集合中的每一项。



你学会了迭代器模式，实在是太好了！
因为我刚刚听说对象村并购公司又完成了
另一项交易……对象村咖啡厅也会合
并进来，供应晚餐菜单。

哇哦，我还以为事情已经够
复杂了，现在要怎么办？



别这样，往好处想。我相信我
们能找到办法来用迭代器模式
解决问题。



看看咖啡厅的菜单

这是咖啡厅的菜单，要把这个CafeMenu类整合进我们的框架，似乎不是太难……看看怎么做。

CafeMenu没有实现我们的新Menu接口，但这很容易修正。

咖啡厅用HashMap存储菜单项。不知道这个有没有支持Iterator？等一下会研究……

```
public class CafeMenu {
    Map<String, MenuItem> menuItems = new HashMap<String, MenuItem>();

    public CafeMenu() {
        addItem("Veggie Burger and Air Fries",
            "Veggie burger on a whole wheat bun, lettuce, tomato, and fries",
            true, 3.99);
        addItem("Soup of the day",
            "A cup of the soup of the day, with a side salad",
            false, 3.69);
        addItem("Burrito",
            "A large burrito, with whole pinto beans, salsa, guacamole",
            true, 4.29);
    }

    public void addItem(String name, String description,
        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.put(name, menuItem);
    }

    public Map<String, MenuItem> getMenuItems() {
        return menuItems;
    }
}
```

和其他菜单类似，菜单项在构造器中初始化。

在这里，我们创建新的菜单项，并把它添加到菜单项HashMap中。

键是项的名称。 值是menuItem对象。

我们不再需要这个。



削尖你的铅笔

在看下一页之前，请快速写下要让这份代码符合我们的框架，我们要对它做的三件事：

1. _____
2. _____
3. _____

重做咖啡厅菜单代码

我们来重做CafeMenu的代码。我们打算实现Menu接口，还需要为存储在HashMap里的值创建迭代器。和用ArrayList时有点不同，我们来看看……

```
public class CafeMenu implements Menu {
    Map<String, MenuItem> menuItems = new HashMap<String, MenuItem>();

    public CafeMenu() {
        // 构造器的代码写在这里
    }

    public void addItem(String name, String description,
                        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.put(name, menuItem);
    }

    public Map<String, MenuItem> getMenuItems() {
        return menuItems;
    }

    public Iterator<MenuItem> createIterator() {
        return menuItems.values().iterator();
    }
}
```

CafeMenu实现Menu接口，因此Waitress可以像其他两个菜单一样用它。

我们使用HashMap，因为这是一个常用的存储值的数据结构。

和之前一样，我们可以去掉getMenuItems()，因此我们不对Waitress暴露菜单项的实现。

我们在这里实现了createIterator()方法。注意，我们不是取得整个HashMap的迭代器，只是值的迭代器。



Code up Close

HashMap比ArrayList稍微复杂一点，因为它支持键和值，但我们依然可以获取值（也就是MenuItem）的迭代器。

```
public Iterator<MenuItem> createIterator() {
    return menuItems.values().iterator();
}
```

首先，我们取得HashMap的值，也就是HashMap中所有对象的集合。

幸运的是这个集合支持iterator()方法，返回一个java.util.Iterator类型的对象。



我们是否违反了最少知识原则？对此我们能做什么？

把咖啡馆菜单添加到女招待

现在是时候修改Waitress以支持我们的新菜单了。现在Waitress期待的是Iterator，因此应该直截了当：

```
public class Waitress {
```

```
    Menu pancakeHouseMenu;
```

```
    Menu dinerMenu;
```

```
    Menu cafeMenu;
```

咖啡厅菜单会和其他菜单一起被传入Waitress的构造器中，我们把它存放在一个实例变量中。

```
public Waitress(Menu pancakeHouseMenu, Menu dinerMenu, Menu cafeMenu) {
```

```
    this.pancakeHouseMenu = pancakeHouseMenu;
```

```
    this.dinerMenu = dinerMenu;
```

```
    this.cafeMenu = cafeMenu;
```

```
}
```

```
public void printMenu() {
```

```
    Iterator<MenuItem> pancakeIterator = pancakeHouseMenu.createIterator();
```

```
    Iterator<MenuItem> dinerIterator = dinerMenu.createIterator();
```

```
    Iterator<MenuItem> cafeIterator = cafeMenu.createIterator();
```

```
    System.out.println("MENU\n---\nBREAKFAST");
```

```
    printMenu(pancakeIterator);
```

```
    System.out.println("\nLUNCH");
```

```
    printMenu(dinerIterator);
```

```
    System.out.println("\nDINNER");
```

```
    printMenu(cafeIterator);
```

```
}
```

我们用这个咖啡厅的菜单作为晚餐菜单。要把它打印出来，我们只需创建迭代器，把它传给printMenu()。搞定！

```
private void printMenu(Iterator iterator) {
```

```
    while (iterator.hasNext()) {
```

```
        MenuItem menuItem = iterator.next();
```

```
        System.out.print(menuItem.getName() + ", ");
```

```
        System.out.print(menuItem.getPrice() + " -- ");
```

```
        System.out.println(menuItem.getDescription());
```

```
    }
```

```
}
```

```
}
```

这里没有变化。

早餐、午餐和晚餐

我们来更新测试程序以确保所有东西能转起来。

```
public class MenuTestDrive {
    public static void main(String args[]) {
        PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
        DinerMenu dinerMenu = new DinerMenu();
        CafeMenu cafeMenu = new CafeMenu();

        Waitress waitress = new Waitress(pancakeHouseMenu, dinerMenu, cafeMenu);

        waitress.printMenu();
    }
```

创建一个 CafeMenu.....

.....然后把它传给女招待。

现在，我们打印时应该看到所有三个菜单。

测试结果如下，看看来自咖啡厅的新餐厅菜单！

```
File Edit Window Help
% java DinerMenuTestDrive
MENU
----
BREAKFAST
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs and toast
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries
Waffles, 3.59 -- Waffles with your choice of blueberries or strawberries

LUNCH
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat
BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad
Hot Dog, 3.05 -- A hot dog, with sauerkraut, relish, onions, topped with cheese
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice
Pasta, 3.89 -- Spaghetti with marinara sauce, and a slice of sourdough bread

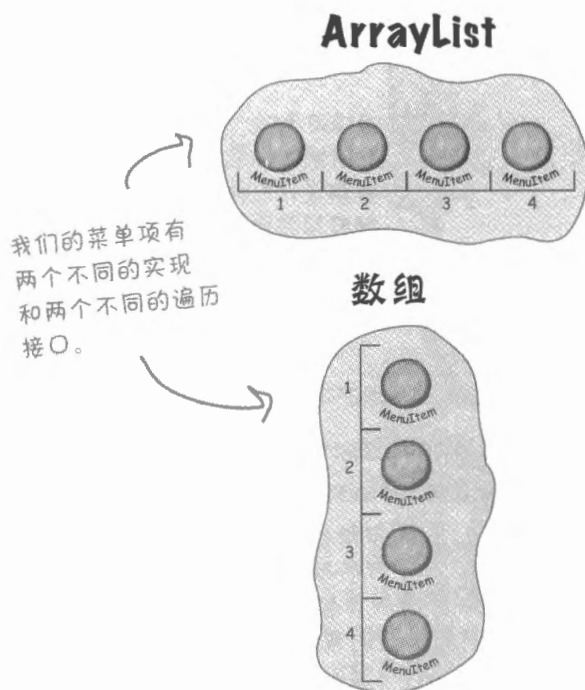
DINNER
Soup of the day, 3.69 -- A cup of the soup of the day, with a side salad
Burrito, 4.29 -- A large burrito, with whole pinto beans, salsa, guacamole
Veggie Burger and Air Fries, 3.99 -- Veggie burger on a whole wheat bun,
lettuce, tomato, and fries
%
```

首先我们遍历煎饼屋菜单。

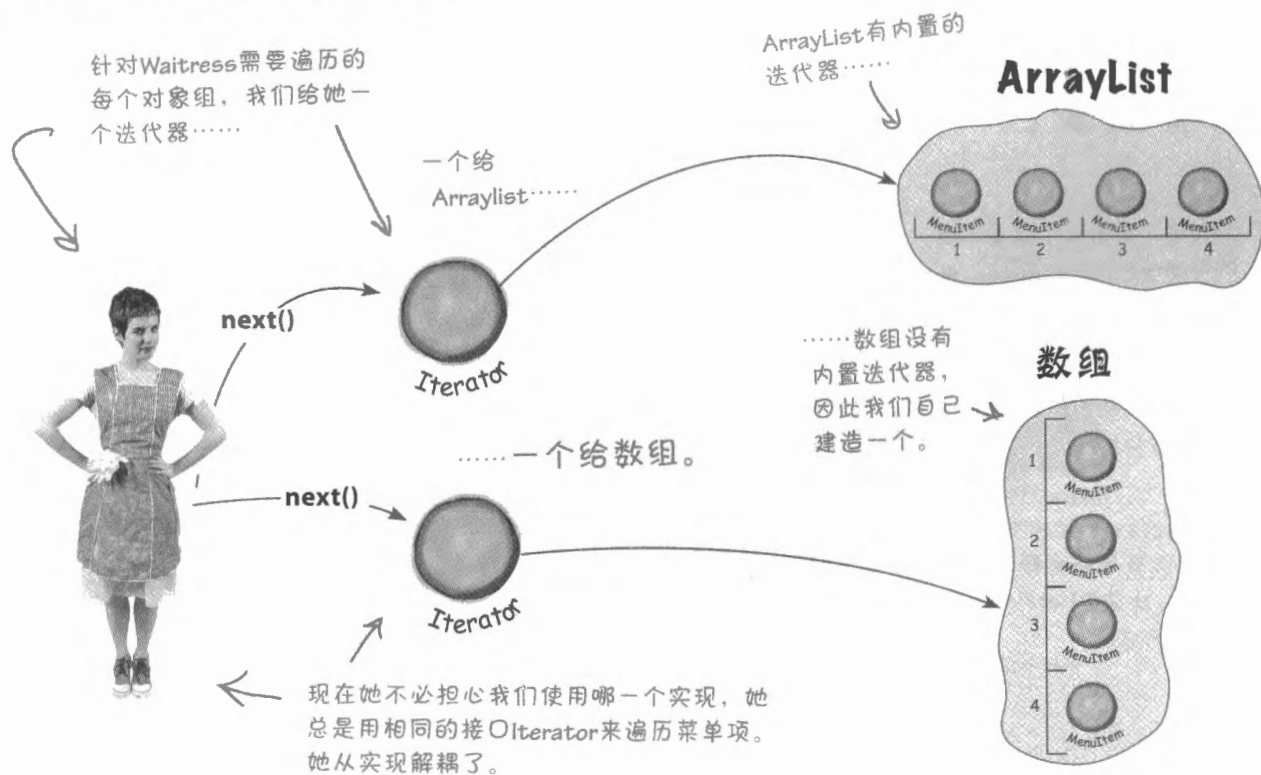
然后是餐厅菜单。

.....最后是咖啡厅菜单，所有遍历代码都一样。

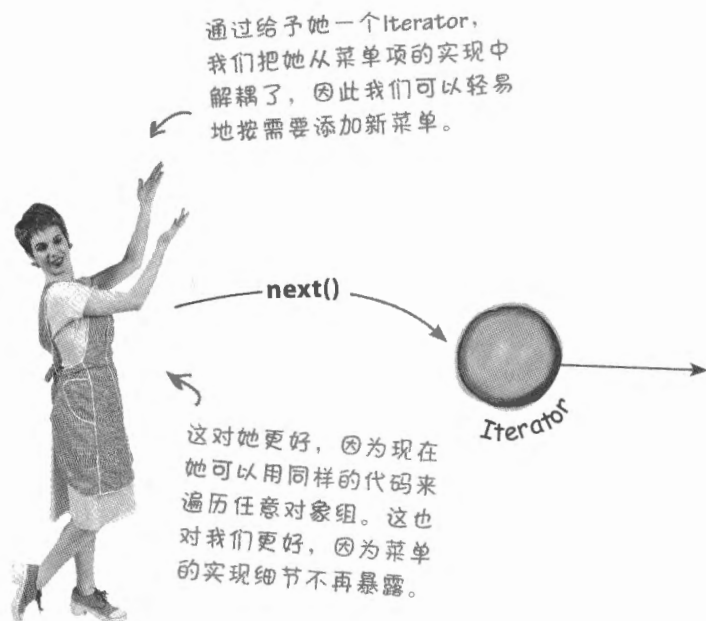
我们做了什么？



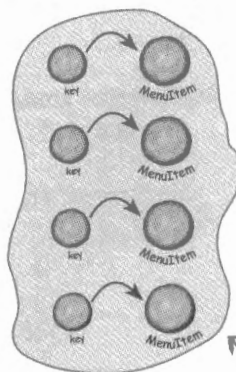
我们把Waitress解耦了……



……让女招待更有扩展性



HashMap



我们很容易地添加另一个菜单项的实现，而且因为我们提供了Iterator，因此女招待知道怎么做。

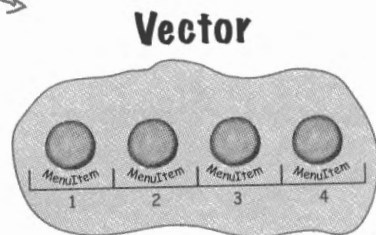
为这个HashMap的值制作一个Iterator很简单，你只要调用values.iterator()就获得一个Iterator。

但还有更多！

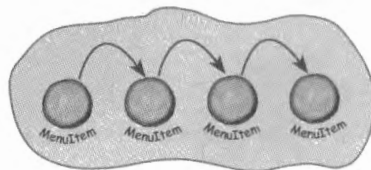
Java给了你很多“集合”类，例如Vector和LinkedList，让你能够存储和检索一组对象。

大多都有不同的接口。

但几乎所有都支持一种获取Iterator的方式。



LinkedList



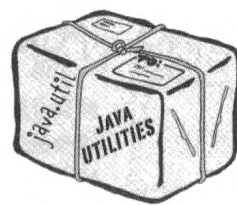
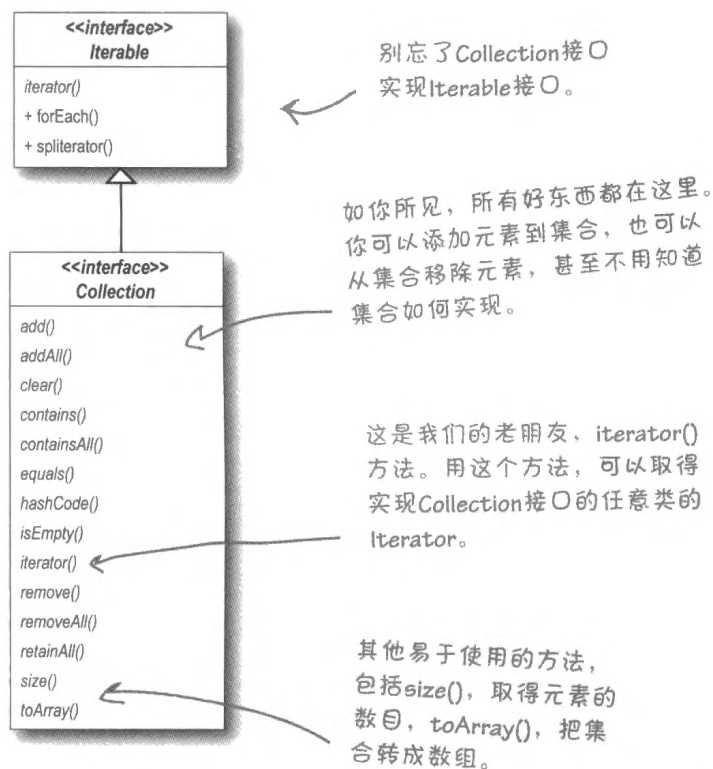
如果它们不支持Iterator，没关系，因为现在你已经知道如何自己建造。

……还有更多！

迭代器和集合

我们用了一些类，这些类是Java Collection Framework的一部分。这个“框架”只是一个类和接口的集合，包括我们用过的ArrayList，还有很多，像Vector、LinkedList、Stack和PriorityQueue。这些类中的每一个都实现java.util.Collection接口，这个接口包含一堆有用的方法，用来操纵一组对象。

我们来快速浏览一下这个接口：



注意！

HashMap是间接支持Iterator的几个类之一。

正如你所看到的，当我们在实现CafeMenu时，可以从中取得一个迭代器，但首先要获取集合的值。仔细想想是说得通的，HashMap持有两组对象key和value。如果我们要遍历value，首先要从HashMap取得它们，然后获得迭代器。



Collection和Iterator的好处是，每个Collection对象都知道如何创建自己的Iterator。调用ArrayList上的iterator()，就返回一个为ArrayList而做的具体的Iterator，不必知道或操心用了哪个具体类。你只要用Iterator接口即可。



代码贴

厨师们决定他们要交替改变午餐的菜单项；换句话说，他们希望在周一、周三、周五和周日提供一些菜单项，在周二、周四和周六提供另一些菜单项。有人已经为新的“轮换”餐厅菜单迭代器编写了代码，但她搞乱了并贴在餐馆冰箱上开玩笑。你能把代码拼回来吗？有一些花括号掉在地板上，因为太小了，不容易捡起来，如果需要你可以自己加上。

```
MenuItem menuItem = items[position];
position = position + 2;
return menuItem;
```

```
import java.util.Iterator;
import java.util.Calendar;
```

```
public Object next() {
```

```
{
```

```
public AlternatingDinerMenuIterator(MenuItem[] items)
```

```
this.items = items;
position = Calendar.DAY_OF_WEEK % 2;
```

```
implements Iterator<MenuItem>
```

```
public void remove() {
```

```
MenuItem[] items;
int position;
```

```
}
```

```
public class AlternatingDinerMenuIterator
```

```
public boolean hasNext() {
```

```
throw new UnsupportedOperationException(
    "Alternating Diner Menu Iterator does not support remove()");
```

```
if (position >= items.length || items[position] == null) {
    return false;
} else {
    return true;
}
```

```
}
```



女招待准备好迎接精彩时刻了吗？

Waitress已经走了很长一段路，但你还得承认，三次调用printMenu()看来有点丑陋。

实事求是，每一次我们添加一个新的菜单，我们不得不打开Waitress实现并添加更多代码。可不可以说这“违反开放关闭原则”？

三次createIterator()调用。

```
public void printMenu() {
    Iterator<MenuItem> pancakeIterator = pancakeHouseMenu.createIterator();
    Iterator<MenuItem> dinerIterator = dinerMenu.createIterator();
    Iterator<MenuItem> cafeIterator = cafeMenu.createIterator();

    System.out.println("MENU\n----\nBREAKFAST");
    printMenu(pancakeIterator);

    System.out.println("\nLUNCH");
    printMenu(dinerIterator);

    System.out.println("\nDINNER");
    printMenu(cafeIterator);
}
```

三次调用
printMenu。

每次我们添加或移除菜单不得不打开这份代码来修改。

这不是女招待的错。我们已经做了很棒的工作，解耦菜单实现，并把遍历行为提取到迭代器。但我们依然把菜单处理为分离和独立的对象，我们需要一种一起管理它们的方式。



Waitress仍然需要调用printMenu()三次，每个菜单一次。你能够想到什么办法把菜单合并，以便只需调用一次或者只传给Waitress一个Iterator就可以遍历所有的菜单？

这不错。我们所要做的只是把这些菜单全部打包进一个ArrayList，然后遍历每个菜单。Waitress的代码就变得简单，可以处理任意数目的菜单。



厨师的话听起来有点道理。我们来试试看：

```
public class Waitress {
    List<Menu> menus;

    public Waitress(List<Menu> menus) {
        this.menus = menus;
    }

    public void printMenu() {
        Iterator<Menu> menuIterator = menus.iterator();
        while(menuIterator.hasNext()) {
            Menu menu = menuIterator.next();
            printMenu(menu.createIterator());
        }
    }

    void printMenu(Iterator<MenuItem> iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }
}
```

现在我们把个菜单列表作为参数，而不是分离的每个菜单。

我们遍历菜单，把每个菜单的迭代器传给重载的printMenu()方法。

这里的代码没有变化。

看起来相当好，虽然我们失去了菜单的名字，但是可以把名字加进每个菜单中。

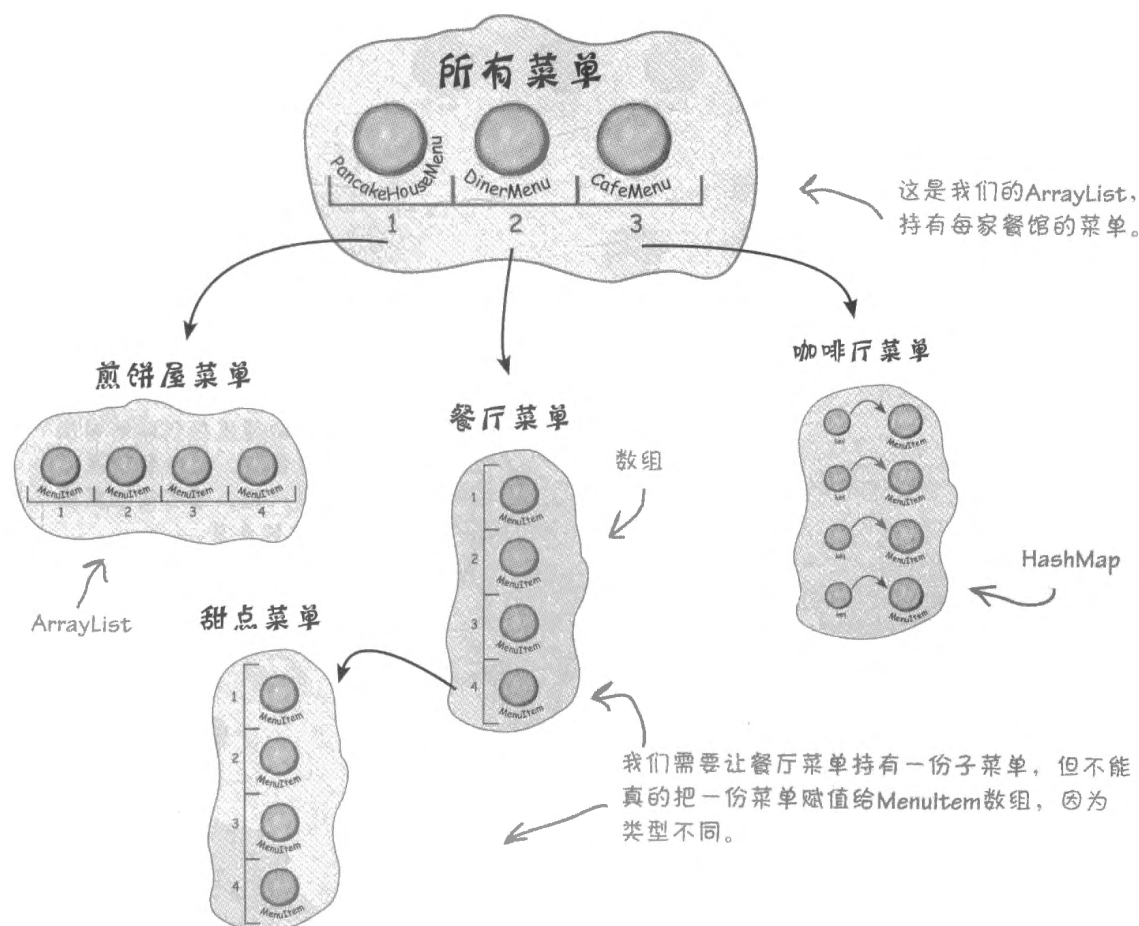
正当我们认为安全的时候……

现在他们要添加一份甜点“子菜单”。

OK，现在怎么办？现在我们不但必须支持多个菜单，而且支持菜单中的菜单。

如果我们能让甜点菜单变成餐厅菜单集合的一个元素，那该多好，但根据现在的实现是做不到的。

我们要的（类似这样）：



但这行不通！

我们不能把甜点菜单赋值给MenuItem数组。

是改变的时候了！

我刚刚听说餐厅要创建一份甜点菜单，并将它插进常规的菜单中。

我们需要什么？

决定重写的时候来临了，我们要重写厨师实现，让它对所有菜单（现在还有子菜单）足够通用。没错，我们打算告诉厨师，重新实现他们的菜单的时刻已经来临。

真实的情况是，我们已经到达了一个复杂度级别，如果现在不重新设计，就无法容纳将来的并购或子菜单等需求。

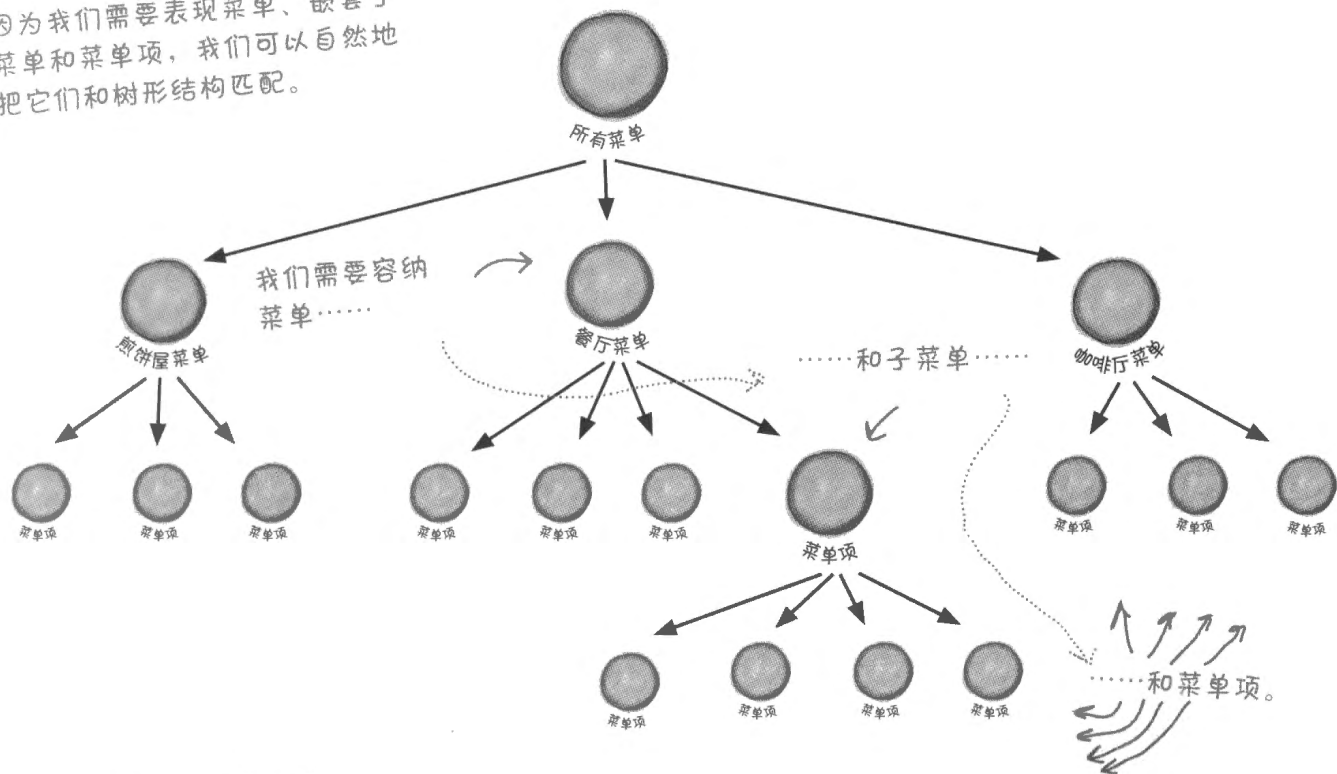
因此，我们的新设计真正需要的什么？

- 我们需要某种树形结构，可以容纳菜单、子菜单和菜单项。
- 我们需要确认我们有办法遍历每个菜单的菜单项，而且至少像现在一样方便。
- 我们可能需要以更有弹性的方式遍历菜单项。例如，我们可能只需要遍历餐馆的甜点菜单，或者我们可能需要遍历餐馆的整个菜单，包括甜点子菜单。

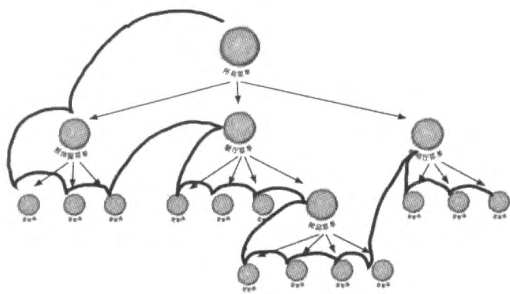
到了为了成长必须重构代码的时候了。如果不这么做，就会导致僵化和没有弹性的代码，没有萌发新生命的希望。



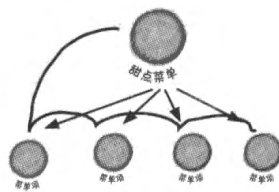
因为我们需要表现菜单、嵌套子菜单和菜单项，我们可以自然地把它和树形结构匹配。



我们仍然需要能遍历树上的所有项。



我们也需要能更弹性地遍历，例如，遍历一份菜单。



你如何处理这个新的设计需求？在翻页之前想一想。

定义组合模式

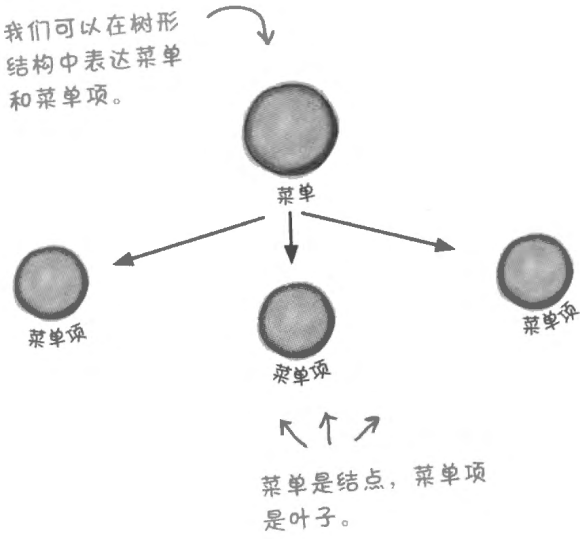
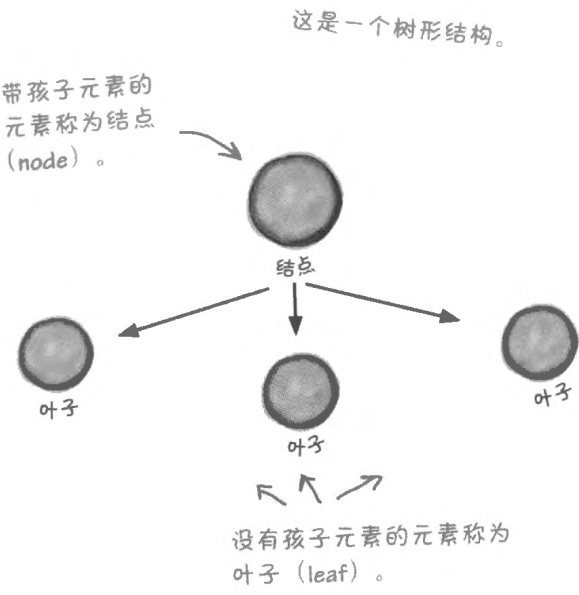
没错，我们打算引进另一个模式来解决这个问题。我们没有放弃迭代器，它依然是我们解决方案的一部分。但是，管理菜单的问题已经到了一个迭代器无法解决的新维度。因此，我们打算往后退，用组合模式（Composite Pattern）解决它。

我们不打算绕来绕去。我们现在就直接给出官方定义：

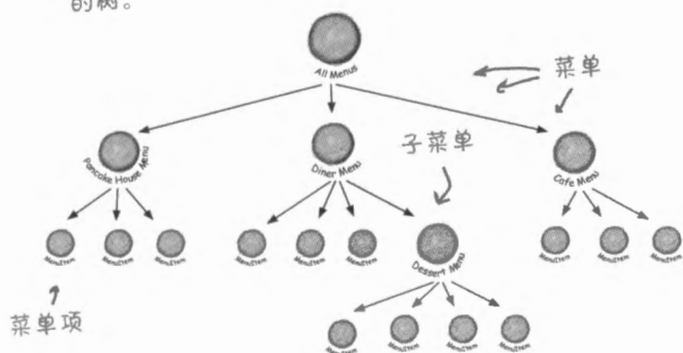
组合模式允许你将对象组合成树形结构来表现部分-整体层次结构。组合让客户可以统一处理个别对象和对象组合。

我们以菜单为例来思考：这个模式让我们可以创建树形结构，嵌套的菜单和菜单项组可以用同一结构来处理。通过把菜单和菜单项放进同一结构，我们创建了一个部分-整体层次，即一颗对象树，它由部件（菜单和菜单项）组成，却可以当成一个整体对待。

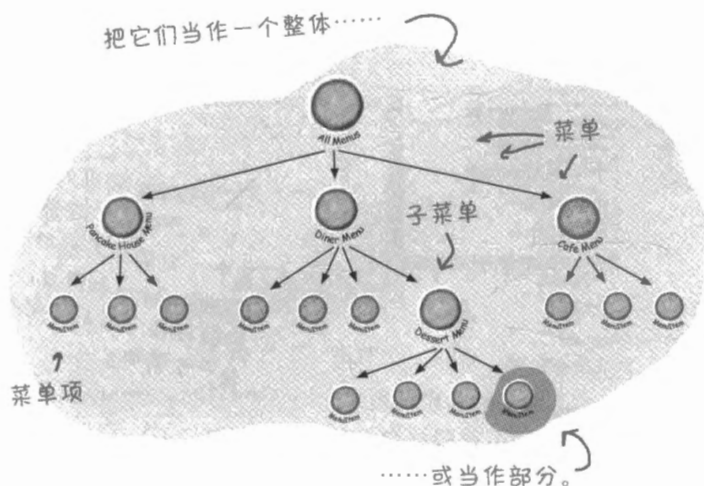
然后，我们就可以使用这个模式来“统一处理个别对象和组合”。这是什么意思？意思是如果我们有了树形结构的菜单、子菜单可能还有子子菜单以及菜单项，那么任何菜单都是一个“组合”，因为它可以包含其他菜单和菜单项。个别对象就是菜单项，它们不持有其他对象。正如你将看到的，使用遵循组合模式的设计能让我们编写一些简单的代码，可以在整个菜单结构应用同样的操作（像打印）。



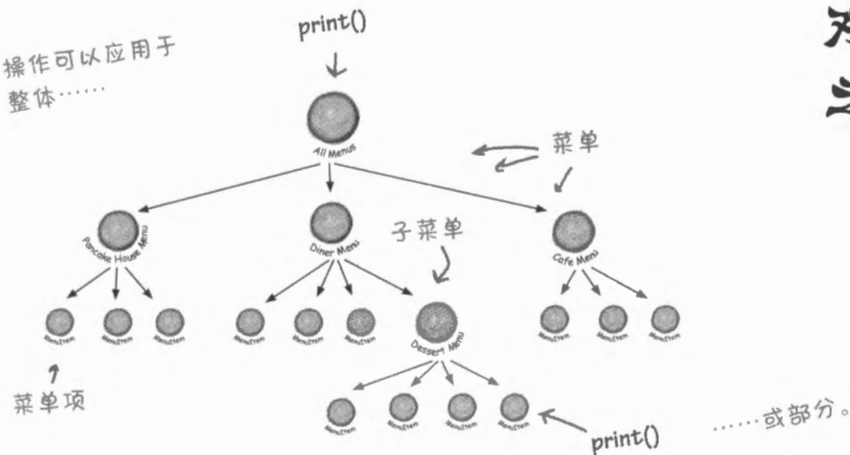
我们可以创建任意复杂的树。



把它们当作一个整体……

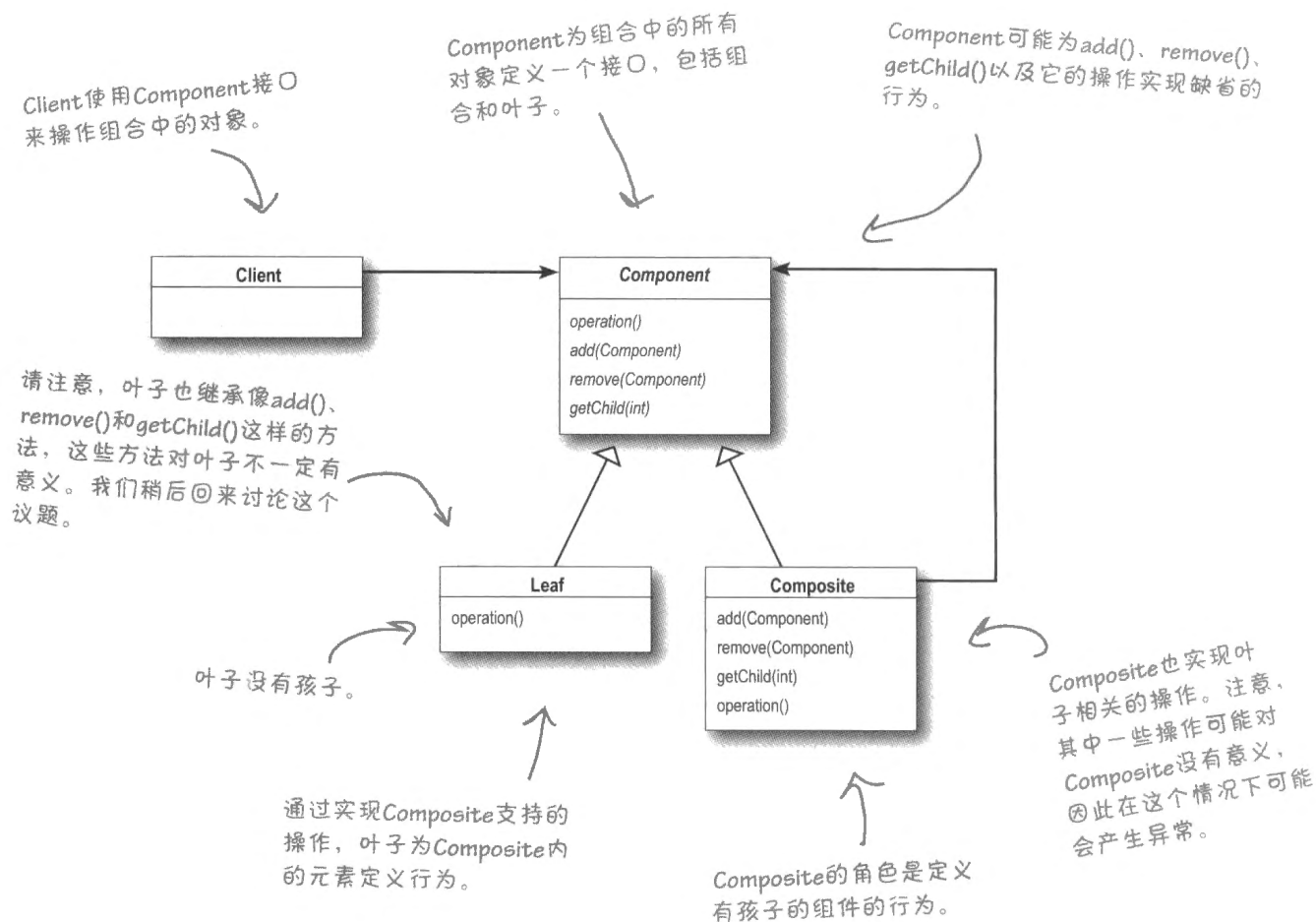


操作可以应用于整体……



组合模式允许我们以树的形式建造对象结构，树的结点包含对象组合和个别对象。

使用组合结构，我们可以对组合和个别对象应用同样的操作。换句话说，在大多数情况下我们可以忽略对象组合和个别对象之间的差别。



没有蠢问题

问：组件、组合、树？我搞混了。

答：组合包含组件。组件有两种：组合和叶元素。听起来像递归？是的。组合持有一组孩子，孩子可能是其他组合或叶元素。

当你用这种方式组织数据时，最终会得到树形结构（确切说是自上而下的树形结构），根部是一个组合，而组合的分支一直延伸到叶子。

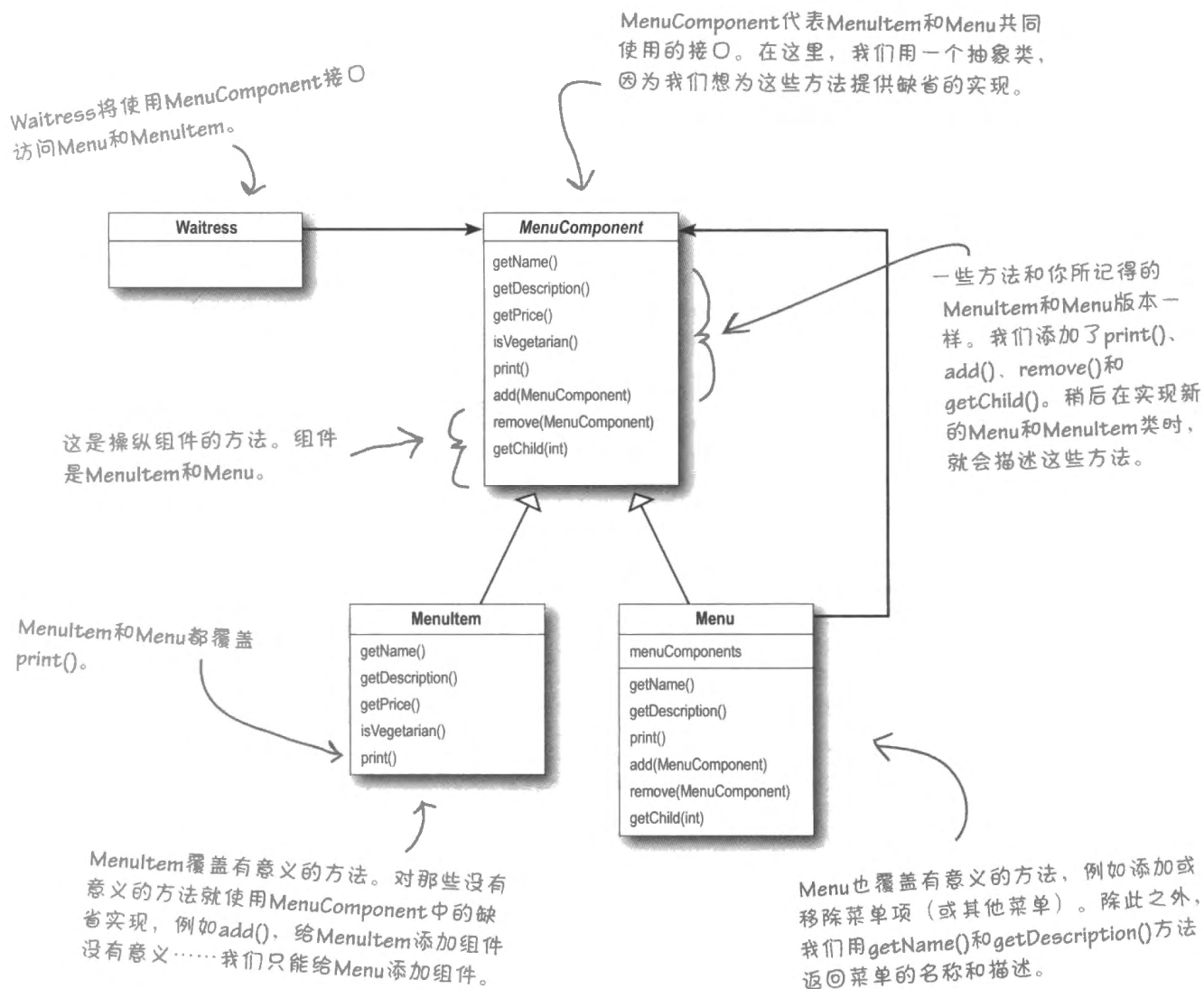
问：这和迭代器有什么关系？

答：记住，我们采取了新的方法。我们打算用新的解决方案来重新实现菜单：组合模式。因此，迭代器到组合并没有什么神奇变换。尽管如此，这两者可以合作无间。你很快就会看到，我们可以在组合的实现中使用迭代器，而且做法还不止一种。

用组合设计菜单

因此，我们如何把组合模式应用到我们的菜单？一开始，我们需要创建一个组件接口，该接口作为菜单和菜单项的通用接口，允许我们统一地对待它们。换句话说，我们可以在菜单或菜单项调用同样的方法。

现在，调用菜单或菜单项上的某些方法可能没有意义，但我们可以处理，只需等一会。目前，我们快速浏览一下菜单如何符合组合模式结构：



实现MenuComponent

OK，我们打算从MenuComponent抽象类开始。记住，菜单组件的角色是为叶子和组合结点提供一个接口。现在你可能想问，“MenuComponent这不是扮演两个角色吗？”可能是这样的，我们稍后再讨论这一点。但是，目前为止我们打算提供方法的一个缺省实现，这样，如果MenuItem（叶子）或Menu（组合）不想实现某些方法（像叶结点的getChild()），可以用一些基本的行为作为后备。

MenuComponent为
每个方法提供缺省
的实现。

```
public abstract class MenuComponent {

    public void add(MenuComponent menuComponent) {
        throw new UnsupportedOperationException();
    }
    public void remove(MenuComponent menuComponent) {
        throw new UnsupportedOperationException();
    }
    public MenuComponent getChild(int i) {
        throw new UnsupportedOperationException();
    }

    public String getName() {
        throw new UnsupportedOperationException();
    }
    public String getDescription() {
        throw new UnsupportedOperationException();
    }
    public double getPrice() {
        throw new UnsupportedOperationException();
    }
    public boolean isVegetarian() {
        throw new UnsupportedOperationException();
    }

    public void print() {
        throw new UnsupportedOperationException();
    }
}
```

所有组件都必须实现MenuComponent接口，但是，因为叶子和结点的角色不同，我们不能总是为每个有意义的方法定义缺省的实现。有时你能做得最好事情是抛出运行时异常。

因为有些方法只对MenuItem有意义，而有些只对Menu有意义，缺省实现是UnsupportedOperationException。这样，如果MenuItem或Menu不支持某个操作，不必做任何事情，只要继承缺省实现。

我们把“组合”方法组织在一起，即添加、移除和取得MenuComponent。

这里是“操作”方法，它们被MenuItem使用。我们也可以把其中一些用在Menu上，再过几页我们展示Menu代码时就会看到。

print()是一个“操作”方法，Menu和MenuItem都会实现它，但我们在这里也提供了缺省操作。

实现MenuItem

OK，我们来看MenuItem类。这是组合类图里的叶子类，它实现组合内元素的行为。

```
public class MenuItem extends MenuComponent {
    String name;
    String description;
    boolean vegetarian;
    double price;
```

首先，我们需要扩展MenuComponent接口。

```
    public MenuItem(String name,
                    String description,
                    boolean vegetarian,
                    double price)
```

构造器需要传入名字、描述等，并保持对它们的引用。这和我们旧的MenuItem实现很像。

```
{
    this.name = name;
    this.description = description;
    this.vegetarian = vegetarian;
    this.price = price;
}
```

```
    public String getName() {
        return name;
    }
```

这是我们的getter方法，和之前的实现一样。

```
    public String getDescription() {
        return description;
    }
```

```
    public double getPrice() {
        return price;
    }
```

```
    public boolean isVegetarian() {
        return vegetarian;
    }
```

这和之前的实现不一样，这里我们覆盖了MenuComponent类中的print()方法。对MenuItem来说，此方法打印完整的菜单条目：名字、描述、价格，以及是否为素食。

```
    public void print() {
        System.out.print(" " + getName());
        if (isVegetarian()) {
            System.out.print("(v)");
        }
        System.out.println(", " + getPrice());
        System.out.println(" -- " + getDescription());
    }
}
```

我很高兴我们朝着这个方向走。我想这会给我需要的弹性，以实现一直想做的可丽饼菜单。



实现组合菜单

现在我们有MenuItem，我们需要的就是组合类，就叫Menu的。记住，组合类可以持有MenuItem或其他Menu。有一些来自MenuComponent的方法，getPrice()和isVegetarian()，Menu类不实现，因为对Menu没太大意义。

Menu也是MenuComponent，
就像MenuItem。

Menu可以有任意数目类型为
MenuComponent的孩子。我们用
一个内部的ArrayList来持有它们。

```
public class Menu extends MenuComponent {
    List<MenuComponent> menuComponents = new ArrayList<MenuComponent>();
    String name;
    String description;
```

```
    public Menu(String name, String description) {
        this.name = name;
        this.description = description;
    }
```

这和我们旧的实现不一样。我们给每个Menu一个名字和一个描述。以前，我们只是靠每个菜单有不同的类。

```
    public void add(MenuComponent menuComponent) {
        menuComponents.add(menuComponent);
    }
```

```
    public void remove(MenuComponent menuComponent) {
        menuComponents.remove(menuComponent);
    }
```

在这里，我们把MenuItem或其他Menu添加到一个Menu中。因为MenuItem和Menu都是MenuComponent，我们只需要一个方法就可以搞定两个。

```
    public MenuComponent getChild(int i) {
        return menuComponents.get(i);
    }
```

```
    public String getName() {
        return name;
    }
```

```
    public String getDescription() {
        return description;
    }
```

这是用来取得名字和描述的getter方法。

注意，我们没有覆盖getPrice()或isVegetarian()，因为这些方法对Menu没有意义（虽然你可能坚持认为isVegetarian()有意义）。如果有人试着在Menu上调用这些方法，就会得到UnsupportedOperationException。

```
    public void print() {
        System.out.print("\n" + getName());
        System.out.println(", " + getDescription());
        System.out.println("-----");
    }
}
```

为了打印Menu，我们打印其名称和描述。



等一下，我不了解`print()`的实现。原本以为应该能够在组合和叶子上应用相同的操作。如果我用这个实现对组合应用`print()`，所得到的只是一个简单的菜单名字和描述，而不是打印出组合内的每个项。

没错。因为Menu是一个组合，包含MenuItem和其他Menu，它的`print()`方法应该打印所包含的每个东西。如果没有打印，我们不得不遍历整个组合并自行打印每个项。这就违背了组合结构的目的。

正如你将看到的，正确地实现`print()`是容易的，因为我们可以让每个组件能够打印自己。这是时髦的递归。我们来看看：

修正`print()`方法

```
public class Menu extends MenuComponent {
    List<MenuComponent> menuComponents = new ArrayList<MenuComponent>();
    String name;
    String description;

    // 构造器代码在这里

    // 其他方法在这里
```

我们需要做的只是改变`print()`方法，让它不只打印Menu本身的信息，也打印Menu的所有组件（其他Menu和MenuItem）的信息。

```
public void print() {
    System.out.print("\n" + getName());
    System.out.println(", " + getDescription());
    System.out.println("-----");
```

看！我们在enhanced for loop的幕后用了迭代器。用它遍历Menu的所有组件……可以是其他Menu，可以是MenuItem。

```
    for (MenuComponent menuComponent : menuComponents) {
        menuComponent.print();
    }
}
```

因为Menu和MenuItem都实现了`print()`，我们只要调用`print()`，剩下的就交给它们了。

注意：如果，在遍历期间，我们遇到另一个Menu对象，它的`print()`方法会开始另一个遍历，依此类推。

测试的准备……

差不多是测试的时候了，但是在这之前我们需要更新Waitress的代码，毕竟，它是所测试代码的主要客户：

```
public class Waitress {
    MenuComponent allMenus;

    public Waitress(MenuComponent allMenus) {
        this.allMenus = allMenus;
    }

    public void printMenu() {
        allMenus.print();
    }
}
```

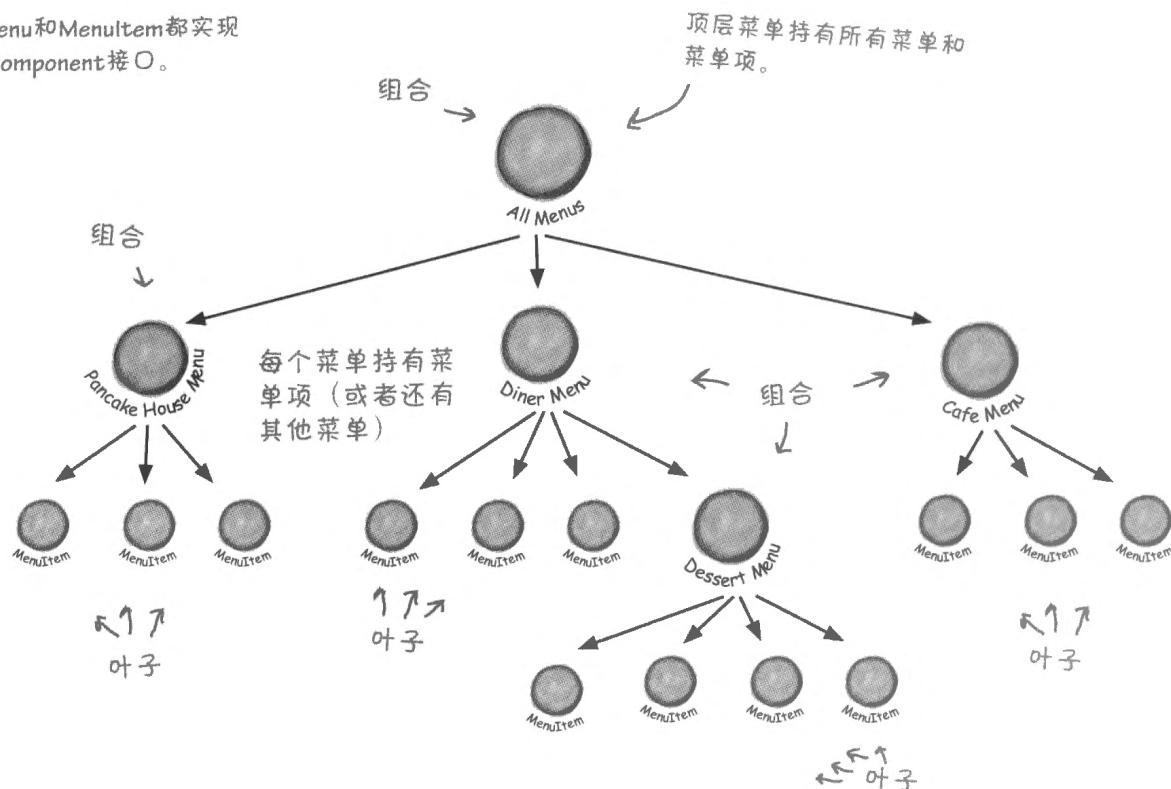
耶！Waitress的代码真的如此简单。现在我们只要把最顶层的菜单组件交给她就可以了，最顶层菜单组件包含其他所有菜单，我们称为allMenus。

她只需要调用最顶层菜单的print()，就可以打印整个菜单层次，所有菜单以及所有菜单项。

我们得到了一个快乐的女招待。

OK，我们编写测试程序之前的最后一件事。我们来了解一下，在运行时菜单组合会是什么样子：

每个Menu和MenuItem都实现MenuComponent接口。



现在轮到测试程序……

OK, 现在我们需要的只有一个测试程序了。不像我们之前的版本, 我们打算在测试程序中处理所有菜单创建。我们可以请每个厨师给我们他的新菜单, 但我们先来测试所有这些。代码如下:

```
public class MenuTestDrive {
    public static void main(String args[]) {
        MenuComponent pancakeHouseMenu =
            new Menu("PANCAKE HOUSE MENU", "Breakfast");
        MenuComponent dinerMenu =
            new Menu("DINER MENU", "Lunch");
        MenuComponent cafeMenu =
            new Menu("CAFE MENU", "Dinner");
        MenuComponent dessertMenu =
            new Menu("DESSERT MENU", "Dessert of course!");
```

我们先创建所有菜单对象。

我们也需要一个顶层菜单, 命名为allMenus。

```
MenuComponent allMenus = new Menu("ALL MENUS", "All menus combined");
```

```
allMenus.add(pancakeHouseMenu);
allMenus.add(dinerMenu);
allMenus.add(cafeMenu);
```

我们使用组合的add()方法来把每个菜单添加到顶层菜单allMenus。

```
// 在这里添加菜单项
```

现在我们需要添加所有菜单项。这是一个例子, 至于其他菜单项, 请看完整的源代码。

```
dinerMenu.add(new MenuItem(
    "Pasta",
    "Spaghetti with Marinara Sauce, and a slice of sourdough bread",
    true,
    3.89));
```

```
dinerMenu.add(dessertMenu);
```

然后我们给菜单添加另一个菜单。dinerMenu在意的只是它持有的每个东西, 无论是菜单项还是菜单, 都是MenuComponent。

```
dessertMenu.add(new MenuItem(
    "Apple Pie",
    "Apple pie with a flakey crust, topped with vanilla ice cream",
    true,
    1.59));
```

给甜点菜单添加一些苹果派……

```
// 在这里添加更多的菜单项
```

```
Waitress waitress = new Waitress(allMenus);
```

```
waitress.printMenu();
```

一旦我们构造了整个菜单层次, 把它整个交给Waitress, 你会发现, 女招待打印菜单简直易如反掌。

```
}
}
```

准备好了，开始测试……

注意：此输出基于完整的源代码。

File Edit Window Help

```
% java MenuTestDrive
```

```
ALL MENUS, All menus combined
```

```
-----
```

```
PANCAKE HOUSE MENU, Breakfast
```

```
-----
```

```
K&B's Pancake Breakfast(v), 2.99
```

```
-- Pancakes with scrambled eggs and toast
```

```
Regular Pancake Breakfast, 2.99
```

```
-- Pancakes with fried eggs, sausage
```

```
Blueberry Pancakes(v), 3.49
```

```
-- Pancakes made with fresh blueberries, and blueberry syrup
```

```
Waffles(v), 3.59
```

```
-- Waffles with your choice of blueberries or strawberries
```

```
DINER MENU, Lunch
```

```
-----
```

```
Vegetarian BLT(v), 2.99
```

```
-- (Fakin') Bacon with lettuce & tomato on whole wheat
```

```
BLT, 2.99
```

```
-- Bacon with lettuce & tomato on whole wheat
```

```
Soup of the day, 3.29
```

```
-- A bowl of the soup of the day, with a side of potato salad
```

```
Hot Dog, 3.05
```

```
-- A hot dog, with sauerkraut, relish, onions, topped with cheese
```

```
Steamed Veggies and Brown Rice(v), 3.99
```

```
-- Steamed vegetables over brown rice
```

```
Pasta(v), 3.89
```

```
-- Spaghetti with marinara sauce, and a slice of sourdough bread
```

```
DESSERT MENU, Dessert of course!
```

```
-----
```

```
Apple Pie(v), 1.59
```

```
-- Apple pie with a flakey crust, topped with vanilla ice cream
```

```
Cheesecake(v), 1.99
```

```
-- Creamy New York cheesecake, with a chocolate graham crust
```

```
Sorbet(v), 1.89
```

```
-- A scoop of raspberry and a scoop of lime
```

```
CAFE MENU, Dinner
```

```
-----
```

```
Veggie Burger and Air Fries(v), 3.99
```

```
-- Veggie burger on a whole wheat bun, lettuce, tomato, and fries
```

```
Soup of the day, 3.69
```

```
-- A cup of the soup of the day, with a side salad
```

```
Burrito(v), 4.29
```

```
-- A large burrito, with whole pinto beans, salsa, guacamole
```

```
%
```

我们的菜单都在这里……只要调用顶层菜单的print()，就可以打印一切。

当打印所有餐厅菜单组件时，新的甜点菜单也被打印。



到底怎么回事？首先你告诉我们“一个类，一个责任”，现在却给我们一个模式，让一个类有两个责任。组合模式不但要管理层次结构，还要执行菜单相关的操作。

这个观察有一定的真实性。我们可以这么说，组合模式用单一责任设计原则来换取透明性。什么是透明性？嗯，通过允许组件接口包含孩子的管理操作和叶子的操作，客户可以统一地对待组合和叶子，因此，一个元素是组合还是叶子结点，对客户来说是透明的。

现在，考虑到我们在Component类中有两种类型的操作，我们失去了一些安全性，因为客户可能会尝试对一个元素做一些不适当或者无意义的事情（像尝试添加菜单到一个菜单项）。这是一个设计决策；我们可以从另一个方向做设计，把责任分离出来放进接口。这么一来，设计上就安全了，元素上任何不合适的调用都会在编译时或运行时被捕获，但我们失去了透明性，我们的代码不得不使用条件语句和instanceof操作符。

因此，回到你的问题，这是一个很典型的折中案例。尽管我们受到设计原则的指导，但是，我们总是需要观察原则对我们的设计造成的影响。我们会故意做一些看似违反原则的事情。但是，在某些情况下，这是一个视角的问题。例如，让孩子管理的操作（像add()、remove()和getChild）出现在叶子中，看起来是不对的，但是换个视角来看，你可以把叶子看作零孩子的结点。



模式曝光

本周访谈：
组合模式，关于实现问题

HeadFirst: 我们今天晚上要和组合模式聊一聊。组合模式，向大家介绍一下自己吧？

组合: 没问题……当你有带整体部分关系的对象的集合，而且要用统一的方式对待这些对象时，就用我这个模式。

HeadFirst: OK，我们来深入了解一下……整体-部分关系是什么意思？

组合: 想象一个图形用户界面（GUI）。你经常会发现有一个顶层的组件，像Frame或者Panel，包含其他组件，像菜单，文本面板、滚动条和按钮。因此你的GUI包含了若干部件，但当你显示它时，一般来说，你认为它是一个整体。你告诉顶层组件显示，然后就由组件显示它的所有部件。我们称包含其他组件的组件为组合对象，称不包含其他组件的组件为叶子对象。

HeadFirst: 这就是你所谓的“用统一的方式对待对象”吧？组合和叶子有共同的方法可以调用？

组合: 没错。我可以告诉组合对象显示或者告诉叶子对象显示，它会做正确的事情。组合对象会告诉它的所有组件显示。

HeadFirst: 这意味着每个对象都有相同的接口。要是组合中存在做不同事情的对象，怎么办？

组合: 为了组合在工作时对客户透明，必须为组合内的所有对象实现同一接口。否则，客户不得不操心每个对象实现了哪个接口，这就违背了组合模式的目的。显然，这也意味着，有时候对某些对象来说，一些方法调用没有意义。

HeadFirst: 那你怎么处理这事情？

组合: 嗯，有一些办法可以应对。有时你可以让这样的方法不做事，或者返回null或false，看哪个适合你的应用。另一些时候，你可能要更主动一些，抛出异常。当然，客户必须乐意多做一些工作，确认方法调用不会做意料之外的事情。

HeadFirst: 但如果客户不知道它们正在处理的对象是哪一种，在不检查类型的情况下，它们怎么知道应该调用什么呢？

组合: 如果你稍微有一点创意，你可以整理一下方法的结构，以便缺省的实现做一些有意义的事情。例如，如果客户正在调用组合上的 getChild()，这是有意义的。如果你把叶子想成没有孩子的对象，这个调用对叶子也有意义。

HeadFirst: 啊……聪明。但我听说，一些客户很担心这个问题，以至于他们需要为不同的对象使用分开的接口，这样就不会产生没有意义的方法调用了。这还是组合模式吗？

组合: 是的，这是更安全版本的组合模式，但这需要客户先检查每个对象的类型，再做方法调用，这样对象可以正确转换类型。

HeadFirst: 告诉我们更多关于组合和叶子对象如何构造的信息。

组合: 通常是树形结构，就是某种层次结构。根是顶层的组合，它的孩子是组合或者叶子。

HeadFirst: 孩子会不会反向指向它们的父亲？

组合: 是的，组件可以有一个指向父亲的指针，以便更容易遍历结构。而且，如果你引用了某个孩子，你想删除它，你需要取得父亲来移除孩子。有指向父亲的引用，这做起来就更容易。

HeadFirst: 在你的实现中，真的有相当多的事情要考虑。在实现组合模式时，还有其他我们应该考虑的问题吗？

组合: 确实有。一个是孩子的次序。要是你的组合需要保持孩子的特定次序怎么办？那么你就需要更精致的管理方案来添加和移除孩子，你也不得不关心如何遍历层次结构。

HeadFirst: 好观点，我没想到过。

组合: 你考虑过缓存吗？

HeadFirst: 缓存？

组合: 是的，缓存。有时，如果组合结构复杂或者遍历的开销太高，那么实现组合结点的缓存是有帮助的。例如，如果你经常遍历一个组合，而且它的所有孩子要计算某些结果，你可以实现一个缓存来临时保存结果，省去遍历的开销。

HeadFirst: 嗯，组合模式比我猜想的有更多东西。在我们结束之前，最后一个问题：你认为你的最强项是什么？

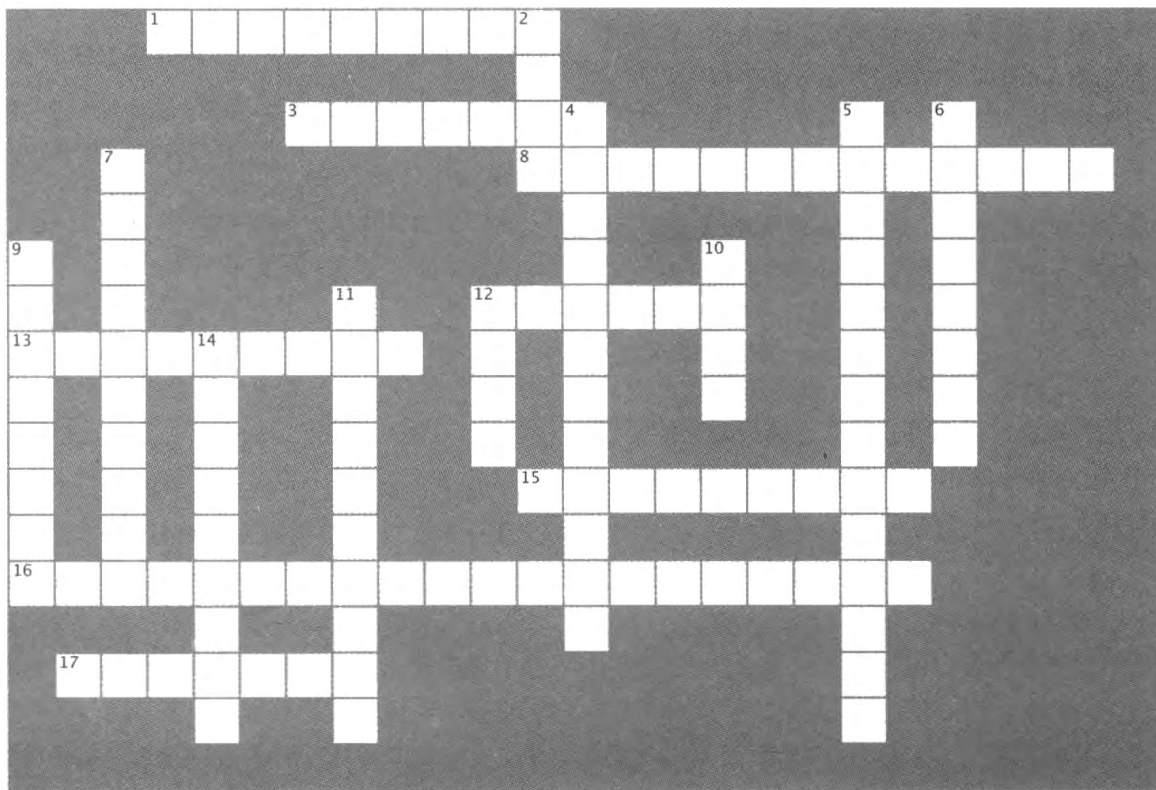
组合: 我肯定认为是我让客户的生活更简单。我的客户不必操心他们处理的是组合对象还是叶子对象，因此他们不必到处编写if语句来确认在正确的对象上调用了正确的方法。通常，他们做一次方法调用并对整个结构执行一次操作即可。

HeadFirst: 这听起来像是一个很重要的好处。毫无疑问，你是一个有用的模式，可以帮助我们收集以及管理对象。时间过了……非常感谢收看，稍后回来看另一个模式告白。



设计模式填字游戏

好好思考这个组合填字游戏。



横排

1. Collection和Iterator在这个包中。
3. 这个类间接支持迭代器。
8. 迭代器通常使用这个模式（两个词）创建。
12. 一个类应该只有一个做这个的原因。
13. 我们封装这个。
15. 用户界面包经常为它们的组件使用这个模式。
16. 陈述每个类只有一个责任的原则名称（两个词）。
17. 这个菜单让我们改变了整个实现。

竖排

2. 没有孩子。
4. 和餐馆合并（两个词）。
5. 迭代器模式从聚合的_____解耦客户。
6. 一个分离对象，可以遍历集合。
7. HashMap的值和ArrayList都实现这个接口。
9. 我们给她做了Java赋能。
10. 组件可以是一个组合或这个。
11. 一个组合持有这些。
12. 第三个被收购的公司。
14. 我们删除PancakeHouseMenuIterator，因为这个类已经提供了迭代器。

★ WHO DOES WHAT? ★

匹配每个模式及其描述:

模式	描述
策略	客户统一对待对象集合和个别对象。
适配器	提供一种遍历对象集合而不暴露集合的实现的方式。
迭代器	简化一群类的接口。
外观	改变一个或多个类的接口。
组合	允许一群对象在某些状态变化时被通知。
观察者	封装可以互换的行为，并使用委托来决定使用哪个行为。



你的设计工具箱中的工具

工具箱多了两个新模式，处理对象集合的两个很棒的方法。

OO原则

封装变化。

组合优于继承。

针对接口编程，不针对实现编程。

为交互对象之间的松耦合设计而努力。

类应该对扩展开放，对修改关闭。

依赖抽象，不要依赖具体类。

只和朋友交谈。

别打电话给我，我会打电话给你。

类应该只有一个变化的原因。

基础

抽象

封装

多态

继承

另一个基于设计变化的重要原则。

OO Patterns

迭代器，提供一种顺序访问聚合元素而不暴露潜在实现的方式

组合，把对象组合进树形结构来表达部分-整体层次。组合让客户能够统一对待个体对象和对象组合。

又是“二合一”的一章。

要点

- 迭代器允许访问聚合的元素，而不暴露其内部结构。
- 迭代器将遍历聚合的工作取出并封装进另一个对象。
- 当使用迭代器时，在支持遍历数据的操作方面，我们减轻了聚合的责任。
- 迭代器提供了一个遍历聚合项的共同接口，当我们编码使用聚合的项时，就可以使用多态。
- Iterable接口让我们能够获取迭代器和使用Java的 enhanced for loop。
- 我们应该致力只为一个类分配一个责任。
- 组合模式允许客户统一地处理组合和个别对象。
- 组合结构内的任意对象称为组件。组件可以是其他组合或者叶子。
- 实现组合模式有许多设计上的折衷。你要根据需要平衡透明和安全。



削尖你的铅笔

答案

基于我们的printMenu()实现，下列哪些是对的？

- ☒ A. 我们在针对PancakeHouseMenu和DinerMenu的具体实现编码，而不是针对接口。
- ☐ B. 女招待没有实现Java女招待API，所以她没有遵守标准。
- ☒ C. 如果我们决定从使用DinerMenu切换到另一种菜单，此菜单的菜单项列表是用哈希表实现的，那么我们不得不修改女招待中的很多代码。
- ☒ D. 女招待需要知道每个菜单如何表达内部的菜单项集合，这违反了封装。
- ☒ E. 我们有重复的代码，printMenu()方法需要两个分离的循环，来遍历两种不同的菜单。如果我们加上第三种菜单，我们又得加一个循环。
- ☐ F. 这个实现没有基于MXML（Menu XML），所以没有办法互操作。



削尖你的铅笔

答案

在看下一页之前，请快速写下要让这份代码符合我们的框架，我们要对它做的三件事：

1. 实现Menu接口

2. 去除getItems()

3. 添加createIterator()并返回一个Iterator，该Iterator可以遍历HashMap值



代码贴答案

还原了的“备选”DinerMenu迭代器。

```
import java.util.Iterator;  
import java.util.Calendar;
```

```
public class AlternatingDinerMenuIterator
```

```
implements Iterator<MenuItem>
```

```
{  
    MenuItem[] items;  
    int position;
```

```
    public AlternatingDinerMenuIterator(MenuItem[] items)
```

```
{  
        this.items = items;  
        position = Calendar.DAY_OF_WEEK % 2;
```

```
    }  
    public boolean hasNext() {
```

```
        if (position >= items.length || items[position] == null) {  
            return false;  
        } else {  
            return true;  
        }  
    }
```

```
    public MenuItem next() {
```

```
        MenuItem menuItem = items[position];  
        position = position + 2;  
        return menuItem;  
    }
```

```
    public void remove() {
```

```
        throw new UnsupportedOperationException(  
            "Alternating Diner Menu Iterator does not support remove()");  
    }
```

注意，这个迭代器实现不支持remove()。

WHO DOES WHAT?

答案

匹配每个模式及其描述：

模式	描述
策略	客户统一对待对象集合和个别对象。
适配器	提供一种遍历对象集合而不暴露集合的实现的方式。
迭代器	简化一群类的接口。
外观	改变一个或多个类的接口。
组合	允许一群对象在某些状态变化时被通知。
观察者	封装可以互换的行为，并使用委托来决定使用哪个行为。



设计模式填字游戏解答

好好思考这个组合填字游戏。答案如下：

